



PAWMAP: A Community-Driven Stray Animal Rescue and Welfare Platform

By

Anavadhya Devi (2548007)

Sreya Anil (2548052)

Under the guidance of

Dr. Asha Unnikrishnan

A Project report submitted in partial fulfillment of the requirements for
the award of degree of Master of Science (Data Science) of CHRIST
(Deemed to be University)

September – 2026



CERTIFICATE

*This is to certify that the report titled **PawMap** is a bona fide record of work done by **Anavadhya Devi (2548007)**, **Sreya Anil (2548052)** of **CHRIST(Deemed to be University)**, Bengaluru, in partial fulfillment of the requirements of IV Trimester M.Sc. (Data Science) during the year 2026-27.*

Head of the Department

Dr. Nimitha John

Project Guide

Dr. Asha Unnikrishnan

Valued-by:

- | | | |
|----|--------------------|-----------------------------------|
| 1. | Name | : Anavadhya Devi |
| | Register Number | : 2548007 |
| | Name | : Sreya Anil |
| | Register Number | : 2548052 |
| | Examination Centre | : CHRIST(Deemed to be University) |
| 2. | Date of Exam | : |

Acknowledgement

We would like to express our sincere gratitude towards all those who have supported us during the course of this project. We would like to extend our heartfelt gratitude towards Dr. Asha Unnikrishnan for her invaluable guidance and encouragement in completing this project. Their suggestions have helped us shape the project to its finest version.

We would also like to express our thanks to the **Coordinator** and The **Head of Department**, Department of Statistics and Data Science, and CHRIST (Deemed to be University), Bangalore India, for providing us with the necessary resources and infrastructure to complete this project. Lastly, we would like to thank everyone whose inputs and feedback have been extremely valuable in finding the right direction to shape the outcome of this project.

Name :

Registration Number:

Name :

Registration Number:

Abstract

India hosts one of the largest stray animal populations in the world, with tens of millions of stray dogs and cats across its cities. Despite active NGO and volunteer efforts, rescue coordination currently relies on fragmented channels such as WhatsApp groups and phone calls, resulting in duplicated responses, missed cases, lost medical histories, low adoption rates, and an absence of data to guide intervention. PawMap is proposed as a community-driven mobile and web platform designed to address these gaps by unifying stray animal reporting, volunteer dispatch, medical record-keeping, lost-and-found matching, and adoption facilitation within a single system. Built on Django REST Framework, React/React Native, PostgreSQL with PostGIS, and supporting services such as Celery, Redis, and Google Maps API, the platform enables geo-tagged case reporting in under a minute, atomic case-claiming with automatic reassignment, caregiver-independent medical records, a weighted temperament safety rating, and a foster-to-adopt pipeline. A data science layer applies DBSCAN spatial clustering to detect abandonment hotspots, performs coverage gap analysis, and computes volunteer reliability scores to prioritise case assignment. Initially scoped for pilot deployment in Bengaluru, PawMap is designed for future scaling to other Indian cities, offering NGOs an evidence-based tool for coordinated, data-driven animal welfare intervention.

Table of Contents

CHAPTER 1 INTRODUCTION	1
1.1 PROBLEM DESCRIPTION.....	1
1.2 EXISTING SYSTEM	2
1.3 PROJECT SCOPE	3
1.4 OBJECTIVES	4
1.5 PURPOSE, SCOPE AND APPLICABILITY	4
1.5.1 PURPOSE.....	4
1.5.2 SCOPE	5
1.5.3 APPLICABILITY	5
1.6 OVERVIEW OF THE REPORT	5
CHAPTER 2 SYSTEM ANALYSIS.....	7
2.1 FUNCTIONAL SPECIFICATIONS	7
2.1.1 FUNCTIONAL REQUIREMENTS	7
2.1.2 NON-FUNCTIONAL REQUIREMENTS	8
2.2 BLOCK DIAGRAM.....	9
2.3 SYSTEM REQUIREMENTS	10
2.3.1 USER CHARACTERISTICS	10
2.3.2 SOFTWARE AND HARDWARE REQUIREMENTS	11
2.3.3 CONSTRAINTS	12
CHAPTER 3 SYSTEM DESIGN.....	13
3.1 SYSTEM ARCHITECTURE	13
3.2 MODULE DESIGN.....	14
3.3 DATABASE DESIGN.....	16
3.3.1 NORMALISATION SUMMARY.....	16
3.3.2 DATA DICTIONARIES	16
3.3.3 ENTITY RELATIONSHIP DIAGRAM	23
3.3.4 DATA FLOW DIAGRAM.....	25
3.4 ALGORITHMS AND DATA STRUCTURES	26
3.4.1 DBSCAN HOTSPOT DETECTION	26
3.4.2 VOLUNTEER RELIABILITY SCORE (VRS)	27

3.4.3 ATOMIC CASE-CLAIMING	28
3.4.4 CELERY AUTO-RELEASE COUNTDOWN	28
3.4.5 TEMPERAMENT SCORE AGGREGATION.....	28
3.5 SYSTEM CONFIGURATION.....	29
3.6 INTERFACE DESIGN.....	29
3.6.1 USER INTERFACE SCREEN DESIGN	29
3.6.2 APPLICATION FLOW / CLASS DIAGRAM	32
3.7 API DESIGN	34
3.8 SECURITY DESIGN	36
3.9 DEPLOYMENT ARCHITECTURE.....	36
3.10 REPORTS DESIGN	37
CHAPTER 4 IMPLEMENTATION.....	38
4.1 CODING STANDARDS	38
4.1.1 CODE FORMATTING	38
4.1.2 NAMING CONVENTIONS.....	38
4.1.3 CODE COMMENTS AND DOCSTRINGS	38
4.1.4 ERROR HANDLING	38
4.1.5 CODE REUSABILITY	39
4.1.6 PERFORMANCE OPTIMISATION.....	39
4.2 CORE IMPLEMENTATION — CODE EXCERPTS	39
4.2.1 ATOMIC CASE CLAIMING — CASES/VIEWS.PY	39
4.2.2 VOLUNTEER PROXIMITY GEOFENCING — PAWMAP_BACKEND/GEO.PY	40
4.2.3 DBSCAN HOTSPOT DETECTION — ANALYTICS/SERVICES.PY ...	41
4.2.4 VOLUNTEER RELIABILITY SCORE — VOLUNTEERS/MODELS.PY	43
4.2.5 CASE AND STATUS LOG MODELS — CASES/MODELS.PY	44
4.3 SCREENSHOTS AND INTERFACE DEMONSTRATIONS	47
CHAPTER 5 TESTING.....	52
5.1 TYPES OF TESTING.....	52
5.1.1 UNIT TESTING	52
5.1.2 INTEGRATION TESTING.....	53
5.1.3 SYSTEM TESTING	53
5.2 TEST CASES.....	54

5.2.1 REGISTRATION AND LOGIN	54
5.2.2 CASE REPORTING AND CLAIMING	54
5.2.3 ADOPTION PIPELINE.....	55
5.2.4 SECURITY AND VALIDATION TESTING.....	55
5.3 TEST REPORTS	56
5.4 UNIT TEST SOURCE CODE	56
5.4.1 UNIT TESTS — CASES MODULE (CASES/TESTS.PY).....	56
5.4.2 UNIT TESTS — VOLUNTEERS MODULE (VOLUNTEERS/TESTS.PY).....	58
5.4.3 UNIT TESTS — ANALYTICS MODULE (ANALYTICS/TESTS.PY) ..	60
CHAPTER 6 CONCLUSION.....	62
6.1 DESIGN AND IMPLEMENTATION ISSUES	62
6.2 ADVANTAGES AND LIMITATIONS.....	63
6.3 FUTURE ENHANCEMENTS	64
CHAPTER A PILOT DATA DESCRIPTION	65
A.1 DATASET OVERVIEW AND FEASIBILITY STUDY	65
A.2 DATASET FIELDS AND DEFINITIONS	65
A.3 SAMPLE WARD CENTROIDS USED IN PILOT DATASET	66
A.4 DBSCAN RUN RESULTS ON PILOT DATASET	66
CHAPTER B REFERENCES.....	68

LIST OF TABLES

Table 1: Comparison of Existing Systems with PawMap Contributions	2
Table 2: Complete Functional Requirements Specification	7
Table 3: Non-Functional Requirements Specification.....	9
Table 4: User Classes and Characteristics	10
Table 5: Software and Hardware Requirements	11
Table 6: Database Normalisation Summary and Design Decisions	16
Table 7: Data Dictionary — User Table	17
Table 8: Data Dictionary — Ward Table.....	17
Table 9: Data Dictionary — AnimalType Table	18
Table 10: Data Dictionary — Case Table.....	18
Table 11: Data Dictionary — CaseStatusLog Table.....	19
Table 12: Data Dictionary — VolunteerReliabilityScore Table.....	19
Table 13: Data Dictionary — Animal Table.....	20
Table 14: Data Dictionary — TemperamentRating Table.....	21
Table 15: Data Dictionary — MedicalRecord Table.....	21
Table 16: Data Dictionary — AdoptionListing Table	21
Table 17: Data Dictionary — AdopterInterest Table	22
Table 18: Data Dictionary — AdoptionRecord Table.....	22
Table 19: Data Dictionary — HotspotCluster Table	23
Table 20: DBSCAN Algorithm Parameters and Justification	26
Table 21: VRS Formula Component Weights and Data Sources	27
Table 22: Primary UI Screens — Inputs, Outputs and Requirements	30
Table 23: Django Model Classes — Attributes and Methods	32
Table 24: Primary REST API Endpoints	34
Table 25: Security Design Decisions and Implementation Details	36
Table 26: Deployment Architecture — Components and Configuration	36
Table 27: Unit Test Cases — Execution Summary	52
Table 28: Functional Test Cases — User Registration and Authentication	54
Table 29: Functional Test Cases — Case Reporting and Volunteer Dispatch	54
Table 30: Functional Test Cases — Adoption and Fostering Pipeline.....	55
Table 31: Security and Boundary Validation Test Cases	55
Table 32: Pilot Dataset — Field Definitions and Usage.....	65
Table 33: BBMP Ward Centroids for Pilot Dataset Geographic Distribution.....	66
Table 34: DBSCAN Pilot Run Summary Statistics.....	67

LIST OF FIGURES

Figure 1: PawMap High-Level System Block Diagram	10
Figure 2: Five-Layer Client-Server System Architecture Diagram	14
Figure 3: Entity Relationship Diagram — All 13 Entities	24
Figure 4: DFD Level 0 — Context Diagram	25
Figure 5: DFD Level 1 — Expanded Process Diagram	26
Figure 6: Case Reporting Form and Active Cases Map Wireframes	31
Figure 7: Adoption listing wireframe (image gallery)	32
Figure 8: Django Model Class Dependency Diagram	34
Figure 9: Case Reporting Form	48
Figure 10: Active Cases Map	48
Figure 11: Volunteer Claim Interface	49
Figure 12: Medical Record Entry Timeline	49
Figure 13: Adoption Listing Cards	50
Figure 14: NGO Admin Dashboard	51
Figure 15: Analytics View	51
Figure 16: Django Test Runner Execution Output Showing 6/6 Tests Passed	53

ABBREVIATIONS

- **API:** Application Programming Interface
- **ASGI:** Asynchronous Server Gateway Interface
- **AWS:** Amazon Web Services
- **BBMP:** Bruhat Bengaluru Mahanagara Palike
- **DBSCAN:** Density-Based Spatial Clustering of Applications with Noise
- **DFD:** Data Flow Diagram
- **DRF:** Django REST Framework
- **ERD:** Entity Relationship Diagram
- **FCM:** Firebase Cloud Messaging
- **GPS:** Global Positioning System
- **JWT:** JSON Web Token
- **MDS:** Master of Data Science
- **NGO:** Non-Governmental Organisation
- **ORM:** Object Relational Mapping
- **PDF:** Portable Document Format
- **PII:** Personally Identifiable Information
- **REST:** Representational State Transfer
- **RBAC:** Role-Based Access Control
- **SDR:** System Design Report
- **SOS:** Save Our Souls (Emergency Escalation)
- **SRS:** Software Requirements Specification
- **TOC:** Table of Contents
- **UUID:** Universally Unique Identifier
- **VRS:** Volunteer Reliability Score
- **WCAG:** Web Content Accessibility Guidelines
- **WGS84:** World Geodetic System 1984 (GPS coordinate standard)

CHAPTER 1

INTRODUCTION

1.1 PROBLEM DESCRIPTION

Urban India harbours an estimated 62 million stray dogs and an indeterminate but substantial population of stray cats, goats, cattle, and other animals that have been abandoned or have never been formally owned. Bengaluru alone, governed by the Bruhat Bengaluru Mahanagara Palike (BBMP), is home to over 300,000 street dogs distributed across its 198 administrative wards. The consequences of unmanaged stray populations are multi-dimensional: daily incidents of territorial aggression threatening public safety, vehicular accidents injuring animals and pedestrians, disease transmission vectors including rabies and leptospirosis, and a persistent cycle of suffering for animals who receive no timely medical attention.

The core challenge, however, is not merely the scale of the problem but the near-complete absence of a coordinated, technology-mediated response infrastructure. When a citizen observes an injured animal, their only recourse is typically to post on a neighbourhood WhatsApp group, share a photograph on social media with a plea for help, or dial one of several NGO helplines that may be overwhelmed, unmanned, or simply unaware of who is geographically closest to the incident. This fragmented workflow introduces four categories of critical failure:

- **Duplicated Response:** Multiple volunteers converge on the same high-visibility case while other cases — less visible on social media — remain entirely unattended for hours or days.
- **Lost Medical Continuity:** Animals treated at one clinic or by one volunteer have no persistent identity. When they are transferred to another carer or clinic, their entire treatment history is lost, forcing caregivers to start from scratch with vaccinations, deworming, and diagnosis.
- **Reactive Instead of Proactive Planning:** NGO administrators and municipal bodies have no spatial intelligence about where abandonments cluster geographically. Sterilisation and vaccination drives are therefore planned on guesswork rather than data, misallocating limited resources.

- **No Accountability or Transparency:** No unified audit trail exists to verify whether a reported case was actually attended, what intervention was provided, and what the outcome was. This inability to measure impact hampers fundraising, policy advocacy, and inter-organisational collaboration.

PawMap is designed as a direct technological response to these four systemic gaps. By integrating atomic case-claiming mechanisms, persistent animal health identities, geospatial clustering analytics, and structured adoption pipelines into a unified platform, PawMap transforms stray animal rescue from an informal, reactive activity into a structured, data-driven civic welfare operation.

1.2 EXISTING SYSTEM

A review of the current landscape reveals that existing tools address only isolated aspects of the stray animal welfare problem. WhatsApp groups and social media platforms serve as the primary coordination channels but impose severe limitations: they have no case management lifecycle, no volunteer dispatch logic, and produce no searchable, structured data. NGO helplines provide a human coordination layer but are limited by staff capacity and operating hours, and they do not integrate with any digital case tracking system.

A systematic literature review covering academic publications, deployed applications, and policy frameworks from 2010 to 2026 identifies the following tools and their limitations:

Table 1: Comparison of Existing Systems with PawMap Contributions

Year	Source / Reference	Core Techniques	Gaps and Drawbacks
2010	WHO & WSPA [9]	Operational guidelines for municipal dog population management; no digital component	Prescriptive policy document; no technology integration; no community-participation mechanism
2021	Bhalla, Kemmers & Vasques [2]	Socio-ecological field study of free-ranging dogs in Bengaluru; ethnographic and static GIS mapping	Descriptive research only; no real-time reporting; no volunteer coordination module
2023	Kochumon et al. [3]	Mobile app for stray dog identification using image recognition; GPS tagging of sightings	No volunteer dispatch; no medical record system; no adoption pipeline; limited to identification
2025	Rajwani et al. [4]	AI-based real-time reporting with image classification for	No geospatial clustering; no volunteer management; standalone

		injury severity detection	reporting without case lifecycle
2025	Subramaniam & Mammi [5]	Stray Rescue Management System (SRMS): centralised NGO case tracking database	No community reporting; no mobile-first interface; no analytics or hotspot mapping capability
2025	Efe, Yildiz & Yakut [6]	Smart city mobile app for stray tracking, adoption listings, and shelter management	No volunteer reliability scoring; no DBSCAN analytics; no foster-to-adopt pipeline design
2025	Karuna Nidhan App [7]	Community rescue app with NGO integration; India-focused; rescue case submission	Closed-source; limited spatial analytics; no temperament safety rating; no NGO reporting tools
2026	Rajkumar et al. [8]	Paw Safe: AI/IoT framework for stray dog rescue, healthcare, and urban governance	Theoretical framework only; no deployed prototype; no spatial clustering or volunteer scoring

PawMap's distinguishing contributions relative to this landscape are four-fold: (1) an atomic volunteer dispatch mechanism with reliability scoring and automated reassignment; (2) a DBSCAN-based geo-intelligence layer calibrated for Indian urban ward geographies; (3) a foster-to-adopt pipeline with aggregated temperament safety scoring; and (4) a platform designed from the outset for NGO co-governance with full audit trails.

1.3 PROJECT SCOPE

PawMap is a cross-platform application accessible on Android, iOS, and web browsers. It enables community members to report stray or injured animals with geo-tagged photographs, connects verified volunteers for rapid geofenced response, maintains caregiver-independent medical records through a persistent Animal ID system, facilitates structured foster-to-adopt pipelines, and generates geospatial intelligence for NGO decision-making. The platform is designed for initial deployment across Bengaluru's BBMP administrative wards, with an architecture that explicitly supports national scale-out through modular app design, containerised deployment, and configuration-driven ward boundaries.

The project scope for MDS482-4 encompasses five core functional modules: Stray Animal Reporting, Volunteer Dispatch and Case Management, Medical Records, Adoption and Fostering, and Analytics and Geospatial Intelligence. Hardware integrations such as IoT/NFC collar tagging, drone-assisted surveys, and direct payment gateways are explicitly deferred to a future phase beyond the academic submission boundary.

1.4 OBJECTIVES

The six formal objectives driving all functional and non-functional requirements of PawMap are defined as follows:

- **Objective 1 — Rapid Mobile Ingestion:** To enable any community member, including unauthenticated guests, to report a stray or injured animal within 30 seconds by submitting a geo-tagged photograph and structured case details through the mobile or web interface.
- **Objective 2 — Atomic Volunteer Dispatch:** To eliminate duplicate and missed responses through an atomic case-claiming system that locks a case to the first volunteer who accepts it at the PostgreSQL row level, with automatic reassignment after 30 minutes of inactivity via a Celery countdown task.
- **Objective 3 — Persistent Medical Records:** To maintain a continuous, caregiver-independent medical record for each rescued animal, identified by a unique persistent Animal ID and accessible in append-only fashion by all verified veterinary professionals and NGO members on the platform.
- **Objective 4 — Safety Aggregation:** To provide a standardised five-point temperament rating system that aggregates multi-volunteer input into a weighted safety indicator, with more recent ratings weighted $1.5\times$ relative to ratings older than 30 days, reducing risk for first responders.
- **Objective 5 — Structured Adoption:** To facilitate structured adoption through a foster-to-adopt pipeline, adopter interest queues, and home-readiness questionnaires that gate Express Interest submissions, matching animals to suitable permanent homes through a verified NGO-administered queue.
- **Objective 6 — Geospatial Analytics:** To generate actionable geospatial intelligence including DBSCAN-derived abandonment hotspot heatmaps (eps=500m, min_samples=5, haversine metric), coverage gap ward layers, Volunteer Reliability Score leaderboards, and automated monthly PDF impact reports.

1.5 PURPOSE, SCOPE AND APPLICABILITY

1.5.1 PURPOSE

This final project report serves as the authoritative technical reference for PawMap's design, implementation, testing, and evaluation. It consolidates content from the

Software Requirements Specification (SRS, June 2026) and the System Design Report (SDR, July 2026), documenting the final delivered system under course MDS482-4, Department of Statistics and Data Science, Christ (Deemed to be University). The document is intended for project supervisors, academic evaluators, NGO stakeholders, and potential open-source contributors.

1.5.2 SCOPE

The scope covers the complete five-layer client-server architecture of PawMap — from the React 18 frontend and React Native mobile application through the Django REST Framework backend API, Celery worker pool, scikit-learn analytics layer, and PostgreSQL/PostGIS spatial database. It documents all 13 normalised database tables, the six primary algorithms (DBSCAN, VRS, atomic claiming, auto-release, reverse geocoding, temperament aggregation), and all 20 primary REST API endpoints. Testing documentation covers unit tests, integration tests, and functional test case matrices derived from actual API behaviour.

1.5.3 APPLICABILITY

PawMap is currently deployed as a development prototype targeting Bengaluru's BBMP wards. The modular, API-first architecture permits adaptation to any Indian city whose municipal authority can supply administrative ward boundary GeoJSON. Beyond India, the platform's core modules — reporting, dispatch, medical records, adoption — are applicable to any urban environment with an active volunteer community and NGO ecosystem, requiring only configuration changes to ward names, coordinate boundaries, and notification templates.

1.6 OVERVIEW OF THE REPORT

The remainder of this report is organised as follows. Chapter 2 (System Analysis) defines the functional and non-functional requirements derived from the SRS, presents the system block diagram, enumerates user characteristics for all five roles, and lists hardware and software requirements alongside design constraints. Chapter 3 (System Design) documents the five-layer system architecture, the eight module designs, all 13 database table schemas with full data dictionaries, the core algorithms (DBSCAN, VRS, atomic claiming), class attributes and ORM relationships, interface design specifications, API endpoint catalogue, security design decisions, and deployment configuration. Chapter 4 (Implementation) covers coding standards, live

code excerpts from the codebase, and screenshot placeholders. Chapter 5 (Testing) presents unit, integration, and system test results, full functional test case matrices, and the complete unit test source code for the Cases, Volunteers, and Analytics modules. Chapter 6 (Conclusion) discusses implementation challenges, limitations, and future enhancement pathways. Appendix A describes the pilot dataset structure and DBSCAN parameters. Appendix B contains the weekly project diary for both team members.

CHAPTER 2

SYSTEM ANALYSIS

2.1 FUNCTIONAL SPECIFICATIONS

2.1.1 FUNCTIONAL REQUIREMENTS

Functional requirements describe the specific behaviours the system must exhibit to satisfy stakeholder needs. They are derived from the SRS (June 2026) following the IEEE Std 830-1998 template. Each requirement is labelled REQ-XX, assigned a priority (High/Medium/Low), and linked to one of the five functional modules. The full set of 24 functional requirements is documented below:

Table 2.1

Table 2: Complete Functional Requirements Specification

Req. ID	Pri.	Module	Requirement Statement
REQ-01	H	Reporting	The system shall allow case submission with a single photograph, auto-populated GPS coordinates, species selection, and severity level in 30 seconds or fewer under normal 4G network conditions.
REQ-02	H	Reporting	The system shall store unsubmitted cases in local device cache if network connectivity is unavailable and sync them automatically when connectivity is restored.
REQ-03	H	Reporting	Each submitted case shall be assigned a unique sequential CASE-xxxx ID, timestamped in UTC, and linked to a reverse-geocoded ward name derived from submitted coordinates.
REQ-04	M	Reporting	The system shall accept photographs up to 10 MB and compress them to under 500 KB before storage to AWS S3 without visible quality loss using Pillow image processing.
REQ-05	M	Reporting	Guest reporters (unauthenticated) shall be able to submit cases but shall not be able to claim, update, or resolve cases.
REQ-06	H	Dispatch	The system shall implement atomic case-claiming using PostgreSQL row-level locking (SELECT FOR UPDATE inside transaction.atomic()) to prevent concurrent claims of the same case.
REQ-07	H	Dispatch	The system shall automatically release an unclaimed or inactive case back to Open status after 30 minutes via a Celery countdown task, logging the release event in CaseStatusLog.
REQ-08	H	Dispatch	Volunteers shall be able to escalate a case to SOS status, which bypasses the reliability-score queue and immediately pushes Firebase FCM alerts to all active NGO administrators.
REQ-09	M	Dispatch	The system shall display active cases with colour-coded severity pins on a Leaflet.js interactive map, filterable by species and status.
REQ-10	M	Dispatch	Every case status transition shall be recorded in CaseStatusLog with the actor's user ID, old status, new status, UTC timestamp, and an optional free-text note.

REQ-11	H	Medical	Each rescued animal shall have a unique persistent Animal ID (ANIM-xxxx) that survives caregiver handovers and foster transfers.
REQ-12	H	Medical	Medical records shall be readable by all verified NGO members and veterinary professionals; write access shall be restricted to Vet and NGO_Admin roles.
REQ-13	H	Medical	The system shall maintain a complete, append-only audit trail of all medical record entries with author user ID and UTC timestamp; no UPDATE or DELETE is permitted on MedicalRecord rows.
REQ-14	M	Medical	The system shall generate vaccination reminder alerts to the current foster or NGO administrator when a next_due_date is within 7 days via FCM push notification.
REQ-15	L	Medical	The system shall support export of an individual animal's full medical history as a formatted PDF document generated by ReportLab.
REQ-16	M	Adoption	The system shall publish adoption listings with a five-point weighted temperament score (recent ratings weighted 1.5×), a medical summary, and current foster information.
REQ-17	M	Adoption	Adopter profiles shall require verified contact information and a completed home-readiness questionnaire before the Express Interest button is activated.
REQ-18	M	Adoption	The system shall maintain an ordered adopter interest queue per animal; the NGO administrator may reorder, approve, or remove adopters from the queue.
REQ-19	L	Adoption	The system shall send automated welfare follow-up survey emails to confirmed adopters at 30 and 90 days post-adoption to collect outcome data.
REQ-20	M	Analytics	The system shall apply DBSCAN spatial clustering (eps=500m, min_samples=5, haversine metric) to geo-tagged case reports weekly to identify abandonment hotspots.
REQ-21	M	Analytics	The system shall perform a PostGIS spatial join between case locations and active volunteer footprints to flag coverage gap zones weekly, updated on the Ward.coverage_gap_flag column.
REQ-22	M	Analytics	The system shall compute a Volunteer Reliability Score (VRS) for each volunteer as: $VRS = 0.4 \times \text{Response_Rate} + 0.4 \times \text{Completion_Rate} + 0.2 \times (1 - \text{Normalised_Response_Time})$, updated daily.
REQ-23	H	Analytics	Only volunteers with $VRS \geq 0.6$ shall be eligible for SOS-tier case notifications; the dispatch queue shall rank all eligible volunteers by VRS descending.
REQ-24	M	Analytics	The system shall auto-generate and email a monthly PDF impact report (case volume, response times, adoption count, hotspot summary) on the first calendar day of each month.

2.1.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements (NFRs) define the quality attributes — performance, security, reliability, usability — that the system must satisfy independently of specific functional behaviours. They are derived from SRS Section 5 and validated against the deployment architecture defined in the SDR.

Table 2.2**Table 3: Non-Functional Requirements Specification**

NFR ID	Category	Requirement Statement
NFR-01	Performance	Case submission (photo upload, record creation, volunteer dispatch) shall complete within 5 seconds on a 4G connection with minimum 10 Mbps upload speed.
NFR-02	Performance	Volunteer dispatch FCM notification shall be delivered within 10 seconds of case submission under normal server load.
NFR-03	Performance	The Django REST API shall support a minimum of 500 concurrent users without response time degradation beyond 2 seconds per request.
NFR-04	Performance	DBSCAN clustering over 10,000 case records shall complete within 60 seconds on the scheduled Celery analytics worker using Ball Tree spatial index.
NFR-05	Performance	Leaflet map tile loading on the volunteer case map shall complete within 3 seconds on a 4G connection.
NFR-06	Safety	Animal aggression level (1–5) and temperament score shall be displayed prominently at the top of every case card visible to volunteers, with colour coding: Green (1–2), Amber (3), Red (4–5).
NFR-07	Safety	SOS-escalated cases shall trigger audio and vibration alerts on the NGO administrator's device in addition to the FCM push notification.
NFR-08	Safety	A volunteer's live GPS location shall not be visible to community reporters or unverified users; location data is restricted to NGO administrators and the volunteer themselves.
NFR-09	Security	All data in transit shall be encrypted using TLS 1.3; all data at rest shall be encrypted using AES-256 at the AWS RDS and S3 storage layer.
NFR-10	Security	User authentication shall use JWT with 60-minute access token expiry and mandatory re-authentication after 7-day refresh token expiry.
NFR-11	Security	Vet and NGO_Admin roles shall require manual verification (is_verified flag set by Platform_Admin) before activation.
NFR-12	Security	Animal photographs shall be watermarked with the PawMap platform logo before display to prevent unauthorised reuse; Pillow watermarking applied in the Celery compression task.
NFR-13	Security	All administrative actions (role changes, record deletions, report access) shall be logged in CaseStatusLog; the log is append-only and retained for 12 months.
NFR-14	Reliability	The backend API shall maintain 99.5% uptime measured monthly, excluding scheduled maintenance windows.
NFR-15	Usability	A first-time reporter shall complete case submission without tutorial in under 30 seconds, validated by pilot user testing with 10 participants.

2.2 BLOCK DIAGRAM

The system block diagram (Fig 2.1) illustrates the high-level architecture of PawMap. The client tier consists of the React 18 + Vite web application serving NGO administrators and veterinary professionals, and the React Native Expo mobile application serving community reporters, volunteers, and adopters. Both clients

communicate exclusively with the Django REST Framework API gateway over HTTPS. The API gateway routes requests to the PostgreSQL/PostGIS relational database, the Redis message broker, and the Celery task worker pool. External services — Firebase Cloud Messaging (FCM) for push notifications, AWS S3 for object storage, and Leaflet.js tile providers for map rendering — are all accessed through the backend layer, never directly from clients.

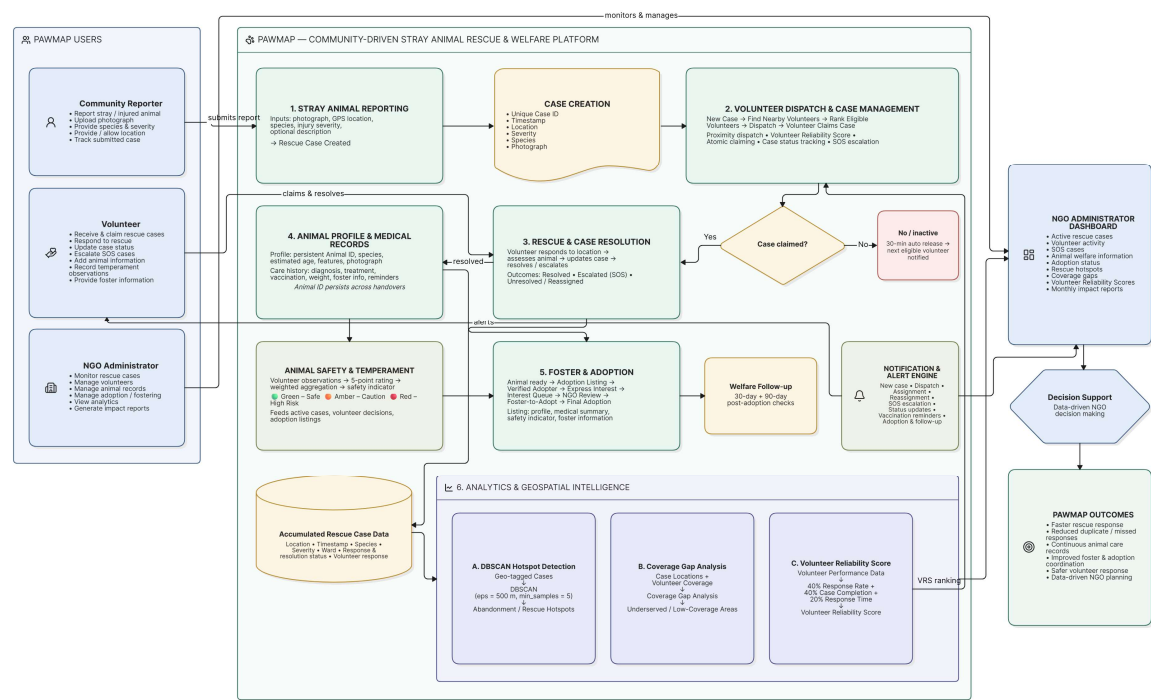


Figure 1: PawMap High-Level System Block Diagram

2.3 SYSTEM REQUIREMENTS

2.3.1 USER CHARACTERISTICS

PawMap serves five distinct user classes, each with different technical proficiency, usage frequency, primary device, and permission scope. The user characterisation table below consolidates information from SRS Section 2.3:

Table 2.3
Table 4: User Classes and Characteristics

User Class	Technical Level	Usage Frequency	Primary Interface	Permissions Summary
Community Reporter (Guest)	Low	One-time / occasional	Mobile app	Submit cases; view public adoption listings only

Registered Reporter	Low-Medium	Occasional	Mobile app	Track own submitted cases; receive resolution notifications
Volunteer	Medium	Regular, location-aware	Mobile app	Claim and update cases; submit temperament ratings; add foster notes
Veterinary Professional	Medium	Infrequent, high-trust	Web dashboard	Read all animal records; create and update medical entries for any animal
NGO Administrator	Medium-High	Frequent, power user	Web dashboard	All Volunteer permissions; manage accounts; publish adoption listings; access analytics

2.3.2 SOFTWARE AND HARDWARE REQUIREMENTS

The following table documents the primary software dependencies and hardware environment requirements for the PawMap platform as deployed for the MDS482-4 semester project:

Table 2.4

Table 5: Software and Hardware Requirements

Component	Version	Purpose	Status
Django + DRF	5.2 / 3.17	Backend API server and ORM	Deployed
React + Vite	18.x / 5.x	NGO Admin and Vet web dashboard	Deployed
React Native (Expo)	SDK 50 / RN 0.73	Cross-platform mobile (Reporter/Volunteer)	In Progress
PostgreSQL + PostGIS	15 / 3.3	Spatial relational database	Deployed
Redis	7.2	Celery message broker and session cache	Deployed
Celery	5.6.x	Background tasks: dispatch, auto-release, analytics	Deployed
scikit-learn	1.9.0	DBSCAN clustering and VRS computation	Deployed
Leaflet.js	1.9.x	Interactive geospatial map rendering	Deployed
Firebase Cloud Messaging	Current	Push notification delivery	Configured
AWS EC2 t3.small	Current	Backend API hosting instance	Provisioned
AWS RDS db.t3.micro	PostgreSQL 15	Managed database service	Provisioned
AWS S3	Current	Photo and PDF object storage	Provisioned
Docker	24.x	Containerisation and deployment	Configured

The mobile application requires Android 10+ or iOS 14+ with minimum 2 GB RAM, a GPS-enabled device, and an active internet connection. The web application

requires Chrome 110+, Firefox 110+, Safari 16+, or Edge 110+. The backend is deployed on Python 3.12+.

2.3.3 CONSTRAINTS

The following design and implementation constraints were identified in SRS Section 2.5 and observed during implementation:

- **Network Degradation:** The mobile application must function with degraded GPS accuracy in dense urban areas and must cache submitted reports locally if connectivity is lost, syncing automatically when connectivity is restored.
- **Encryption:** All PII (email, phone) must be encrypted at rest using AES-256 and in transit using TLS 1.3. Implemented via django-encrypted-model-fields and AWS RDS encryption.
- **Data Protection:** All personal data handling must comply with applicable Indian digital data protection standards.
- **Technology Stack:** The stack is constrained to Django, React, PostgreSQL/PostGIS, Celery/Redis, Leaflet.js, Firebase, and scikit-learn. No proprietary mapping SDK is required.
- **Timeline:** The 16-week project timeline constrains scope to the five core modules; IoT/NFC tag integration and AI-based injury severity detection are deferred.
- **PostGIS Composite FK:** Django ORM does not natively support composite foreign keys; the species+breed relationship to AnimalType is enforced at the model validation layer in clean().
- **DBSCAN Minimum Density:** DBSCAN requires a minimum of min_samples=5 cases in a 500m radius to form a valid cluster; wards with fewer than 5 sightings per run will show no hotspot polygon.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

PawMap follows a five-layer client-server architecture as defined in the System Design Report (SDR, July 2026). All inter-layer communication is over HTTPS (TLS 1.3) using RESTful JSON conventions. Real-time case status updates are delivered over WebSocket connections via Django Channels. External services interface exclusively with the backend layer.

Layer 1 — Users: Four actor types interact with the platform: Community Reporter, Volunteer, NGO Administrator, and Platform Administrator. Layer 2 — Frontend: The mobile application (React Native / Expo) serves Reporters and Volunteers. Primary screens include the Case Reporting Form with photo capture and GPS auto-fill, the Active Cases Map with real-time Leaflet pins, the Volunteer Dashboard with claim controls and auto-release countdown, and the Adoption Listing screen. The web application (React 18 + Bootstrap 5 + Vite) serves NGO Administrators and Veterinary Professionals, providing the NGO Admin Dashboard, Analytics View with DBSCAN heatmap and coverage gap layer, Report Generation, and Adopter Management Panel. Layer 3 — Backend API: The Django REST Framework API server handles JWT authentication, case lifecycle management, atomic volunteer claiming via PostgreSQL row-level locking, medical record access control, adoption queue management, and notification dispatch via FCM. Celery workers execute all asynchronous tasks: photo compression, reverse geocoding via Nominatim, the 30-minute auto-release countdown, weekly DBSCAN scheduling, daily VRS computation, and monthly PDF impact report generation. Layer 4 — Data: PostgreSQL 15 with PostGIS 3.3 stores all structured and spatial data. Key PostGIS functions used are ST_DWithin for volunteer proximity dispatch within 5 km, ST_ClusterDBSCAN for hotspot detection, and ST_MakePoint for indexing case coordinate columns. Redis 7.2 serves as the Celery task broker and Django session cache. AWS S3 stores compressed animal photographs and generated PDF reports. Layer 5 — Data Science: Scheduled Celery tasks invoke Python + scikit-learn processes. The weekly DBSCAN job clusters geo-tagged case coordinates and writes

hotspot cluster records. The daily VRS job recomputes reliability scores for every active volunteer. The spatial join job updates coverage gap flags on the Ward table.

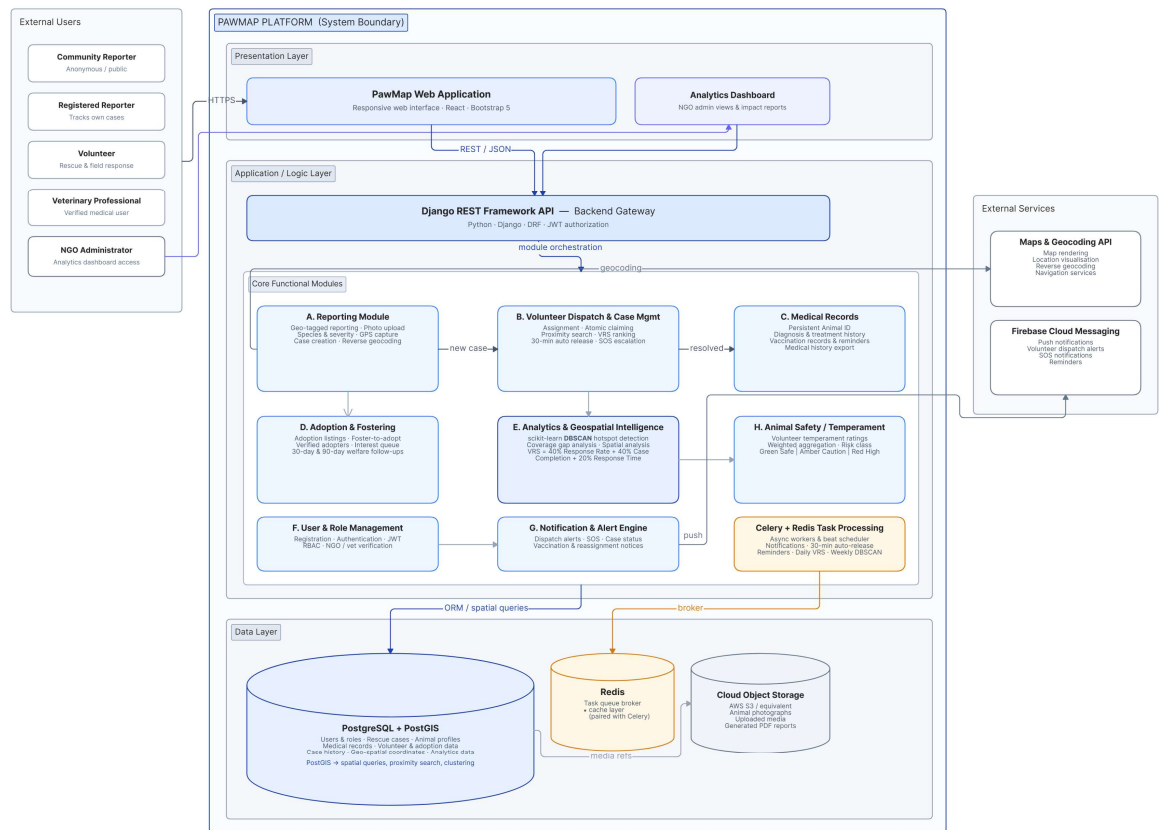


Figure 2: Five-Layer Client-Server System Architecture Diagram

3.2 MODULE DESIGN

The system design partitions the functional requirements across eight operational modules. Each module is responsible for a clearly bounded set of business operations and communicates with other modules only through the API layer or the shared database, maintaining loose coupling.

- **Module 1 — Reporting:** Ingests GPS-tagged sighting reports with compressed photo uploads via AWS S3, performs reverse geocoding to assign BBMP ward names using Nominatim, generates sequential CASE-xxxx IDs, and publishes volunteer dispatch jobs to the Celery queue. Also manages LostPetReport and FoundPetReport for owned pet tracking.
- **Module 2 — Volunteer Dispatch:** Queries Redis for volunteer GPS locations within 5 km of each case, ranks candidates by VRS score descending, dispatches FCM push alerts, manages atomic case claims via SELECT FOR UPDATE, starts

the 30-minute Celery auto-release countdown, and handles SOS escalation broadcasts to all NGO admins.

- **Module 3 — Medical Records:** Generates persistent ANIM-xxxx Animal IDs on rescue case resolution, maintains append-only MedicalRecord entries with vet credentials and timestamps, gates write access to Vet and NGO_Admin roles, triggers vaccination reminder FCM alerts when next_due_date is within 7 days, and supports PDF export via ReportLab.
- **Module 4 — Adoption & Rehoming:** Publishes animal adoption listings with weighted temperament scores, gates Express Interest on completion of the home-readiness questionnaire, maintains ordered AdopterInterest queues per listing, processes NGO Admin approval to create AdoptionRecords, and schedules 30-day and 90-day welfare follow-up survey emails.
- **Module 5 — User & Role Management:** Manages user registration, JWT token issuance and refresh, RBAC permission enforcement across all five roles, and manual verification workflows for Vet and NGO_Admin accounts requiring Platform Admin approval.
- **Module 6 — Notification Engine:** Manages FCM device token registration, composes and delivers case alert payloads for dispatch, SOS broadcasts, vaccination reminders, and adoption queue notifications, and tracks CaseNotification response records per volunteer for VRS input.
- **Module 7 — Analytics Module:** Runs weekly DBSCAN hotspot detection on accumulated case coordinates, performs PostGIS spatial joins for coverage gap analysis, recomputes daily VRS snapshots for all active volunteers, and generates monthly PDF impact reports distributed by email on the first calendar day of each month.
- **Module 8 — Animal Safety:** Aggregates temperament ratings from multiple volunteers using the $1.5\times$ recency weighting formula, updates materialised Animal.temperament_score on each new rating insert, and displays colour-coded safety indicators on all case cards and adoption listings.

3.3 DATABASE DESIGN

3.3.1 NORMALISATION SUMMARY

The database schema was derived through systematic normalisation of the pilot dataset fields and the functional requirements documented in the SRS. The following table summarises the normalisation decisions made prior to constructing the Entity Relationship Diagram (ERD):

Table 3.1

Table 6: Database Normalisation Summary and Design Decisions

Normal Form	Violation Found	Resolution Applied
1NF	Medical records stored as flat repeating columns (diagnosis_1, treatment_1...) per animal	Separated into standalone MedicalRecord table — append-only, one row per clinical entry per animal
1NF	Multiple adopters per animal cannot be represented as flat columns on AdoptionListing	Separated into AdopterInterest table with one row per adopter per listing; queue_position manages order
2NF	Reporter demographic attributes (age_group, prior_experience) depend only on reporter_id, not on case_id	Moved to User table; pilot survey fields retained only in the CSV export, not in production schema
2NF	Ward attributes (centroid coordinates, boundary polygon) depend only on ward_name, not on case_id	Created Ward reference table; ward_name stored on Case as a deliberate denormalisation for query speed
3NF	animal_name is functionally determined by species (e.g., Indian pariah dog maps only to Dog)	Created AnimalType lookup table with composite PK (species, breed); eliminates transitive dependency
3NF	vrs_score is transitively derived from response_rate, completion_rate, avg_response_time_min	All four columns retained in VolunteerReliabilityScore for formula auditability and historical snapshots
3NF	adoption_status is a property of the listing lifecycle, not an intrinsic property of the animal	Moved to AdoptionListing.status; Animal retains only the is_adopted boolean derived from AdoptionRecord
Deliberate Denorm.	ward_name on Case is derivable from coordinates via reverse geocoding	Stored at submission time (Nominatim reverse geocode) for query performance; documented as materialised field
Deliberate Denorm.	Animal.temperament_score derivable from TemperamentRating rows	Stored as materialised weighted average; recomputed on each new TemperamentRating insert via model save()

3.3.2 DATA DICTIONARIES

All 13 normalised database tables are defined below with full data dictionaries. Column names follow snake_case convention; entity names follow PascalCase. FK

denotes Foreign Key; PK denotes Primary Key. ENUM values are drawn from the Django model choices definitions in the codebase.

Table 3.2

Table 7: Data Dictionary — User Table

Column Name	Data Type	Constraints	Description
user_id	VARCHAR(20)	PK	Sequential ID, format USR-0001; assigned on registration
full_name	VARCHAR(100)	NOT NULL	User's full name as entered at registration
email	VARCHAR(150)	UNIQUE, NOT NULL	Login credential and notification email address
phone	VARCHAR(15)	UNIQUE, Nullable	Contact number for NGO coordination; optional at registration
password_hash	VARCHAR(255)	NOT NULL	bcrypt-hashed password; plain text never stored or logged
role	ENUM	NOT NULL	Reporter / Volunteer / Vet / NGO_Admin / Platform_Admin
is_verified	BOOLEAN	DEFAULT FALSE	Set TRUE by Platform_Admin; required for Vet and NGO_Admin activation
is_active	BOOLEAN	DEFAULT TRUE	Soft-delete flag; deactivated accounts cannot log in
created_at	TIMESTAMP	NOT NULL	UTC timestamp of account creation
last_login	TIMESTAMP	Nullable	UTC timestamp of most recent successful authentication

Table 3.3

Table 8: Data Dictionary — Ward Table

Column Name	Data Type	Constraints	Description
ward_name	VARCHAR(100)	PK	BBMP ward administrative name; stable government reference value
city	VARCHAR(100)	NOT NULL	Default 'Bengaluru'; parameterised for future city expansion
centroid_lat	DECIMAL(9,6)	NOT NULL	Ward centroid latitude (WGS84 decimal degrees)
centroid_lon	DECIMAL(9,6)	NOT NULL	Ward centroid longitude (WGS84 decimal degrees)
boundary_geom	GEOMETRY(Polygon,4326)	Nullable	PostGIS polygon from BBMP GeoJSON; used in coverage gap ST_DWithin joins
coverage_gap_flag	BOOLEAN	DEFAULT FALSE	Set TRUE by Analytics worker when ward has unresolved cases with no volunteer

			response within 3 km in the past 7 days
--	--	--	---

Table 3.4**Table 9: Data Dictionary — AnimalType Table**

Column Name	Data Type	Constraints	Description
species	VARCHAR(50)	PK (composite part 1)	Top-level classification: Dog, Cat, Cow, Goat, Horse, Pig, Bird, Other
breed	VARCHAR(100)	PK (composite part 2)	Specific breed or common name, e.g. Indian Pariah Dog, Tabby Cat. Eliminates transitive dependency identified in 3NF analysis.

Table 3.5**Table 10: Data Dictionary — Case Table**

Column Name	Data Type	Constraints	Description
case_id	VARCHAR(20)	PK	Sequential CASE-xxxx format; assigned in save() override
timestamp_utc	TIMESTAMP	NOT NULL	UTC time of report submission (auto_now_add)
latitude	DECIMAL(10,6)	NOT NULL	GPS latitude of sighting (WGS84)
longitude	DECIMAL(10,6)	NOT NULL	GPS longitude of sighting (WGS84)
location_geom	GEOMETRY(Point,4326)	NOT NULL	PostGIS Point generated from lat/lon via trigger; GIST-indexed for ST_DWithin proximity queries
ward	FK -> Ward	SET_NULL	Reverse-geocoded ward assignment; denormalised for query performance
species	VARCHAR(50)	NOT NULL	Observed species: Dog, Cat, Cow, etc.
breed	VARCHAR(100)	DEFAULT 'Unknown'	Breed or type; validated via AnimalType lookup
estimated_age	ENUM	NOT NULL	Puppy / Kitten / Calf / Juvenile / Adult / Senior
severity	SMALLINT	CHECK(1-5), NOT NULL	Reporter-assigned injury severity; 1=healthy, 5=critical
aggression_level	SMALLINT	CHECK(1-5), NOT NULL	Observer-assessed aggression; 1=calm, 5=highly aggressive
injury_type	VARCHAR(200)	NOT NULL	Free-text description of observed injury or condition
bystander_action	ENUM	NOT NULL	none / fed / contacted_ngo
description	TEXT	Blank allowed	Optional extended description of the situation
photo_url	VARCHAR(500)	Blank allowed	S3/GCS path to compressed

			photograph (≤500 KB)
photo	ImageField	Nullable	Local file upload path (upload_to='case_photos/')
status	ENUM	NOT NULL, DEFAULT 'Open'	Open / In_Progress / On_Site / Resolved / Escalated / Unresolved
reporter_id	FK -> User	SET_NULL, Nullable	Sighting submitter; NULL for guest submissions
volunteer_id	FK -> User	SET_NULL, Nullable	Assigned responder; set on atomic claim
response_time_min	DECIMAL(6,1)	Nullable	Minutes between created_at and volunteer claim; VRS input
celery_task_id	VARCHAR(255)	Nullable	Active auto-release countdown Celery task ID; revoked on status update
created_at	TIMESTAMP	NOT NULL	Case creation timestamp (auto_now_add)
updated_at	TIMESTAMP	NOT NULL	Last modification timestamp (auto_now)

Table 3.6**Table 11: Data Dictionary — CaseStatusLog Table**

Column Name	Data Type	Constraints	Description
log_id	VARCHAR(20)	PK	Sequential LOG-xxxx format; append-only (no UPDATE or DELETE permitted via API)
case	FK -> Case	CASCADE, NOT NULL	The case whose status changed; logs cascade-delete with case
actor	FK -> User	SET_NULL, Nullable	User who triggered the transition; NULL for system-generated transitions (auto-release)
old_status	VARCHAR(20)	NOT NULL	Status value before the transition
new_status	VARCHAR(20)	NOT NULL	Status value after the transition
changed_at	TIMESTAMP	NOT NULL	UTC time of status change (auto_now_add)
note	VARCHAR(500)	Blank allowed	Optional free-text note from volunteer or NGO Admin
is_auto_release	BOOLEAN	DEFAULT FALSE	TRUE when triggered by Celery auto-release task, not a human actor

Table 3.7**Table 12: Data Dictionary — VolunteerReliabilityScore Table**

Column Name	Data Type	Constraints	Description
vrs_id	VARCHAR(20)	PK	Sequential VRS-xxxx format
volunteer	FK -> User	CASCADE,	Volunteer this daily score snapshot

		NOT NULL	belongs to
score_date	DATE	NOT NULL, UNIQUE with volunteer	One row per volunteer per day; duplicate prevented by unique_together constraint
cases_notified	SMALLINT	NOT NULL	FCM dispatch notifications sent to this volunteer; denominator for response rate
cases_responded	SMALLINT	NOT NULL	Cases this volunteer claimed via CaseNotification.responded_at; numerator for response rate
cases_completed	SMALLINT	NOT NULL	Claimed cases marked Resolved; numerator for completion rate
avg_response_time_min	DECIMAL(6,1)	NOT NULL	Mean minutes between FCM notification and volunteer claim for all responded cases
response_rate	DECIMAL(4,3)	NOT NULL	cases_responded / cases_notified; stored for formula auditability
completion_rate	DECIMAL(4,3)	NOT NULL	cases_completed / cases_responded; stored for formula auditability
vrs_score	DECIMAL(4,3)	NOT NULL	$0.4 \times RR + 0.4 \times CR + 0.2 \times (1 - \text{norm_RT})$; capped at 0 floor via $\min(\text{norm_RT}, 1)$
flag_for_review	BOOLEAN	DEFAULT FALSE	Set TRUE when VRS < 0.4 for 3 consecutive snapshots; triggers NGO Admin alert

Table 3.8**Table 13: Data Dictionary — Animal Table**

Column Name	Data Type	Constraints	Description
animal_id	VARCHAR(20)	PK	Sequential ANIM-xxxx format; persists through all caregiver handovers
species	VARCHAR(50)	NOT NULL	Species derived from linked Case at rescue resolution
breed	VARCHAR(100)	NOT NULL	Breed or type; linked to AnimalType for validation
estimated_age	ENUM	NOT NULL	Puppy / Kitten / Calf / Juvenile / Adult / Senior
distinguishing_features	TEXT	Nullable	Free-text description of markings, size, colours, or other identifiers
photo_url	VARCHAR(500)	NOT NULL	S3/GCS path to the animal's profile photograph
temperament_score	DECIMAL(3,2)	Nullable	Materialised weighted average; recomputed on each TemperamentRating insert
is_adopted	BOOLEAN	DEFAULT FALSE	Set TRUE when the linked AdoptionRecord is marked completed

created_at	TIMESTAMP	NOT NULL	UTC profile creation timestamp (set when volunteer resolves the linked case)
------------	-----------	----------	--

Table 3.9**Table 14: Data Dictionary — TemperamentRating Table**

Column Name	Data Type	Constraints	Description
rating_id	VARCHAR(20)	PK	Sequential RAT-xxxx format
animal	FK -> Animal	CASCADE, NOT NULL	The animal being rated
volunteer	FK -> User	NOT NULL	Volunteer submitting the rating; only Volunteer-role users may write
score	SMALLINT	CHECK(1-5), NOT NULL	1=calm and friendly, 5=highly aggressive; aggregated into temperament_score
rated_at	TIMESTAMP	NOT NULL	UTC time of rating submission; used to apply 1.5× recency weight for ratings > 30 days old

Table 3.10**Table 15: Data Dictionary — MedicalRecord Table**

Column Name	Data Type	Constraints	Description
record_id	VARCHAR(20)	PK	Sequential MED-xxxx format; append-only (no UPDATE or DELETE via API)
animal	FK -> Animal	CASCADE, NOT NULL	The animal this record belongs to
vet	FK -> User	NOT NULL	Veterinary professional or NGO Admin who created the entry
entry_type	ENUM	NOT NULL	diagnosis / vaccination / treatment / weight
details	TEXT	NOT NULL	Free-text clinical details: diagnosis, vaccine batch, treatment protocol, or weight in kg
entry_at	TIMESTAMP	NOT NULL	UTC time of record creation (auto_now_add)
next_due_date	DATE	Nullable	Next vaccination or follow-up due date; within 7 days triggers FCM vaccination reminder

Table 3.11**Table 16: Data Dictionary — AdoptionListing Table**

Column Name	Data Type	Constraints	Description
-------------	-----------	-------------	-------------

listing_id	VARCHAR(20)	PK	Sequential LST-xxxx format
animal	FK -> Animal	OneToOne, PROTECT	One active listing per animal; PROTECT prevents animal deletion while listing exists
published_by	FK -> User	NOT NULL	NGO Admin who published the listing; only NGO_Admin role may publish via API permission check
published_at	TIMESTAMP	NOT NULL	UTC timestamp of listing publication (auto_now_add)
status	ENUM	NOT NULL, DEFAULT 'Active'	Active / Pending / Completed / Withdrawn

Table 3.12**Table 17: Data Dictionary — AdopterInterest Table**

Column Name	Data Type	Constraints	Description
interest_id	VARCHAR(20)	PK	Sequential INT-xxxx format
listing	FK -> AdoptionListing	CASCADE, NOT NULL	The listing this interest belongs to; cascades with listing deletion
adopter	FK -> User	NOT NULL	Authenticated user expressing interest; UNIQUE together with listing_id prevents duplicate interest
queue_position	INTEGER	NOT NULL	Ordered position in adopter queue; NGO Admin may reorder or remove
home_readiness_complete	BOOLEAN	DEFAULT FALSE	TRUE when adopter has completed home-readiness questionnaire; gates Express Interest button
expressed_at	TIMESTAMP	NOT NULL	UTC timestamp of interest submission (auto_now_add)

Table 3.13**Table 18: Data Dictionary — AdoptionRecord Table**

Column Name	Data Type	Constraints	Description
adoption_id	VARCHAR(20)	PK	Sequential ADO-xxxx format
listing	FK -> AdoptionListing	OneToOne, PROTECT	One completed adoption record per listing maximum
adopter	FK -> User	NOT NULL	The approved adopter selected by NGO Admin from the interest queue
approved_at	TIMESTAMP	NOT NULL	UTC timestamp of NGO Admin approval (auto_now_add)
completed_at	TIMESTAMP	Nullable	UTC timestamp of adoption completion; NULL while in Adoption_Pending state

survey_30d_sent	BOOLEAN	DEFAULT FALSE	Set TRUE when 30-day welfare follow-up survey email is dispatched by Celery task
survey_90d_sent	BOOLEAN	DEFAULT FALSE	Set TRUE when 90-day welfare follow-up survey email is dispatched by Celery task

Table 3.14**Table 19: Data Dictionary — HotspotCluster Table**

Column Name	Data Type	Constraints	Description
cluster_id	VARCHAR(20)	PK	Sequential CLU-xxxx format; one row per cluster per DBSCAN run
run	FK -> HotspotRun	NOT NULL	The analytics run that produced this cluster
run_date	DATE	NOT NULL	Date of the DBSCAN analytics job execution
cluster_label	INTEGER	NOT NULL	DBSCAN output label (0,1,2...); -1 noise points are never stored
centroid_lat	DECIMAL(9,6)	NOT NULL	Mean latitude of all cases in this cluster (WGS84)
centroid_lon	DECIMAL(9,6)	NOT NULL	Mean longitude of all cases in this cluster (WGS84)
case_count	SMALLINT	NOT NULL	Number of case records contributing to this cluster
dominant_species	VARCHAR(50)	NOT NULL	Most frequent species among cases in the cluster; used for hotspot disaggregation on analytics map

3.3.3 ENTITY RELATIONSHIP DIAGRAM

The ERD (Fig 3.2) covers all 13 entities. They are organised into two logical groups: the Case-and-Platform group (User, Ward, AnimalType, Case, CaseStatusLog, CaseNotification, VolunteerReliabilityScore) and the Animal-and-Adoption group (Animal, TemperamentRating, MedicalRecord, AdoptionListing, AdopterInterest, AdoptionRecord, HotspotCluster). The cross-boundary relationship is Case.animal → Animal.animal_id, set post-resolution when the volunteer creates the animal profile.

All cardinalities and cascade behaviours are shown using standard ERD notation.

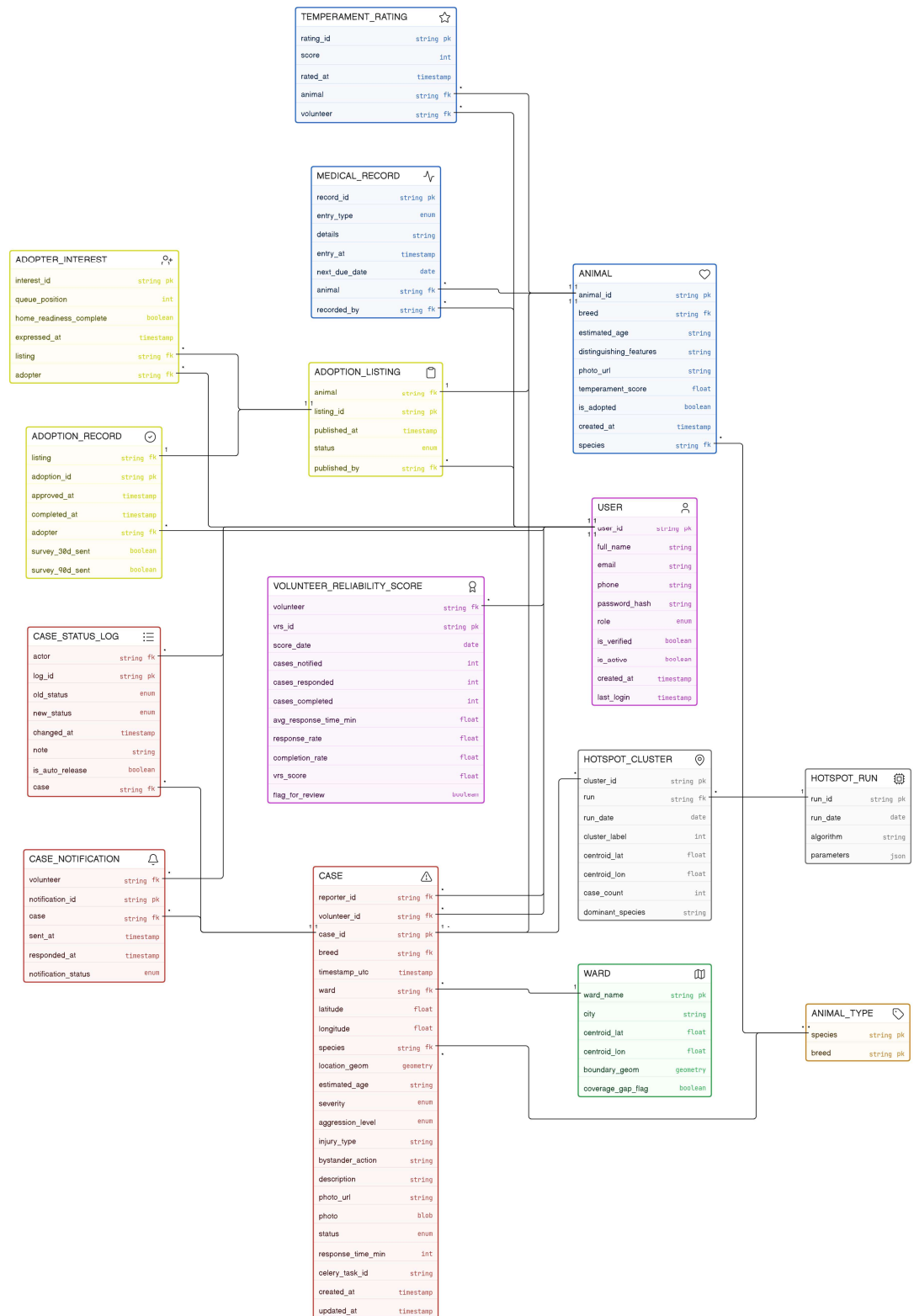


Figure 3: Entity Relationship Diagram — All 13 Entities

3.3.4 DATA FLOW DIAGRAM

The Data Flow Diagram (DFD) describes how sighting data flows through the system from the moment a reporter submits a case to the point that DBSCAN analytics are generated. DFD Level 0 (Context Diagram, Fig 3.3) shows the system as a single process bounded by its external actors: Community Reporter, Volunteer, NGO Admin, Veterinary Professional, and Firebase FCM. DFD Level 1 (Fig 3.4) expands the system boundary to reveal the five major processes: Case Ingestion, Volunteer Dispatch, Medical Record Management, Adoption Pipeline, and Analytics Engine.

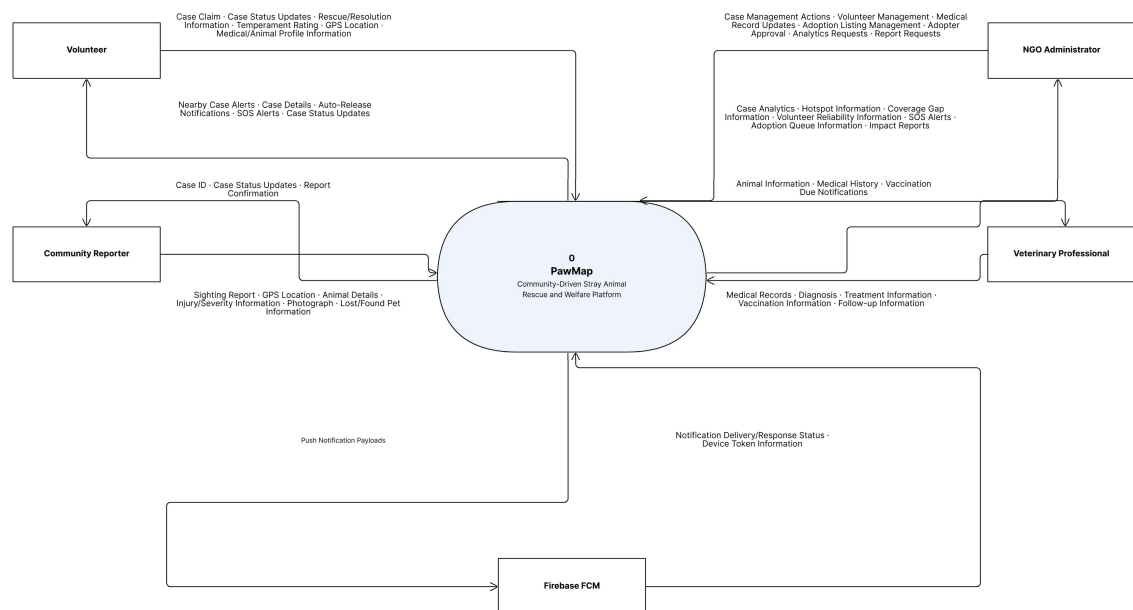


Figure 4: DFD Level 0 — Context Diagram

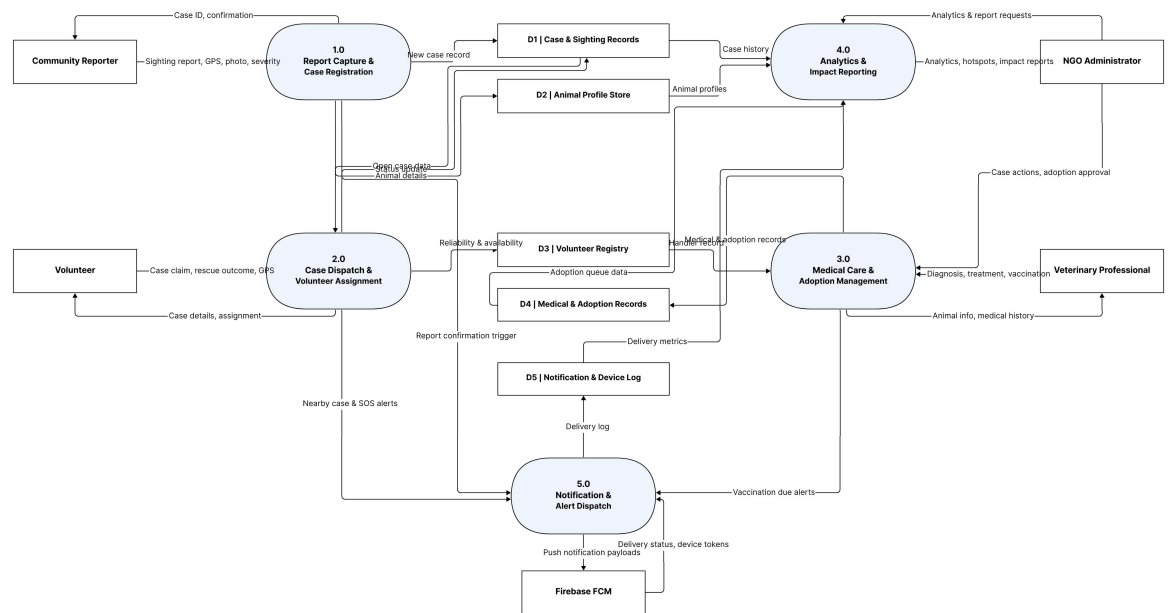


Figure 5: DFD Level 1 — Expanded Process Diagram

3.4 ALGORITHMS AND DATA STRUCTURES

The following subsections document the four primary algorithms implemented in PawMap, including their mathematical specifications, parameter justifications, and implementation details.

3.4.1 DBSCAN HOTSPOT DETECTION

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) identifies statistically significant stray animal abandonment hotspots from accumulated geo-tagged case reports. It is selected over centroid-based methods (k-means) for three reasons: (1) it does not require specifying the number of clusters in advance; (2) it handles clusters of arbitrary geographic shape — critical for Bengaluru's irregular ward boundaries; and (3) it explicitly labels low-density points as noise (-1), preventing false hotspot identification in areas of high reporter density without genuine abandonment clustering.

Table 3.15

Table 20: DBSCAN Algorithm Parameters and Justification

Parameter	Value	Justification
eps (neighbourhood radius)	500 m	Corresponds to approximately one Bengaluru urban ward block. Cases within 500 m are spatially proximate for NGO

		intervention planning.
min_samples	5	Minimum 5 cases required to form a core point. Prevents single-incident locations from being classified as hotspots.
metric	haversine	Accounts for Earth's curvature. Required for geographic coordinate inputs (lat/lon converted to radians before input to scikit-learn).
algorithm	ball_tree	O(n log n) time complexity with Ball Tree spatial index; handles 10,000 case records within 60 seconds on the Celery worker.
Input structure	numpy array (n,2)	Each row is [latitude_radians, longitude_radians] for a resolved case. eps converted from 500m to radians: eps_rad = 500/6371000.
Output	cluster labels array	Integer label per case: -1=noise (not stored); 0,1,2...=cluster ID. Stored in HotspotCluster with centroid and case_count.

The mathematical conversion of eps from metres to radians follows:

```
eps_rad = eps_m / 6371000.0 # Earth radius = 6,371,000 metres
coords_rad = np.radians(coords) # Convert (lat, lon) array to radians
```

3.4.2 VOLUNTEER RELIABILITY SCORE (VRS)

The VRS is a weighted composite metric computed daily per volunteer by the Celery scheduler. It ranks volunteers in the dispatch queue and gates SOS-tier case eligibility (VRS ≥ 0.6 required per REQ-23). The formula is:

```
VRS = 0.4 × Response_Rate + 0.4 × Completion_Rate + 0.2 × (1 -
Normalised_Response_Time)
```

Where:

```
Response_Rate      = cases_responded / cases_notified
Completion_Rate    = cases_completed / cases_responded
Normalised_RT      = min(avg_response_time_min / 30.0, 1.0) #
capped at 1
```

The 30-minute normalisation baseline corresponds to the auto-release threshold (REQ-07). Capping norm_RT at 1.0 ensures the VRS never goes below 0.0. A volunteer with VRS < 0.4 for three consecutive daily snapshots triggers flag_for_review = TRUE and an NGO Admin alert per SRS Business Rule 5.5.

Table 3.16

Table 21: VRS Formula Component Weights and Data Sources

Component	Weight	Formula	Data Source
Response Rate	40%	cases_responded / cases_notified	CaseNotification table: notifications sent vs. responded_at set
Completion Rate	40%	cases_completed / cases_responded	Case.status = Resolved for cases claimed by this volunteer

Avg Response Time	20%	$1 - \min(\text{avg_response_time_min} / 30, 1)$	Case.response_time_min; baseline = 30 min (auto-release threshold)
VRS Score	—	$0.4 \times \text{RR} + 0.4 \times \text{CR} + 0.2 \times (1 - \text{norm_RT})$	Stored daily in VolunteerReliabilityScore with all component values

3.4.3 ATOMIC CASE-CLAIMING

To prevent two volunteers simultaneously claiming the same case (a race condition inherent in distributed systems), PawMap uses PostgreSQL row-level locking via SELECT FOR UPDATE inside a serialisable transaction at the Django ORM layer:

```
with transaction.atomic():
    case = Case.objects.select_for_update().get(case_id=case_id)
    if case.status != 'Open':
        return Response({'error': f'Case is already {case.status}'},
                        status=HTTP_409_CONFLICT)
    case.volunteer = request.user
    case.status = 'In_Progress'
    case.response_time_min = round((now - case.created_at).seconds
    / 60, 1)
    case.save()
```

If a second claim arrives while the first transaction holds the PostgreSQL row lock, the database blocks the second transaction until the first commits. The second transaction then reads status = 'In_Progress' and returns HTTP 409 Conflict. This guarantees exactly-one assignment without any application-layer race conditions or optimistic locking overhead.

3.4.4 CELERY AUTO-RELEASE COUNTDOWN

When a volunteer claims a case, a Celery countdown task is created with a 30-minute delay. The task ID is stored on Case.celery_task_id. If the volunteer updates the case status before 30 minutes elapse, the countdown task is revoked via celery.app.control.revoke(). If the timer fires, the Celery worker atomically resets Case.status to Open, clears Case.volunteer_id, writes a CaseStatusLog entry with is_auto_release=TRUE, and re-triggers the volunteer dispatch flow. The underlying data structure is a Redis-backed Celery beat scheduler with task metadata stored as a Redis hash keyed by task ID.

3.4.5 TEMPERAMENT SCORE AGGREGATION

The five-point temperament score is computed as a weighted average of all TemperamentRating entries for a given animal. Ratings submitted more than 30 days

ago receive weight 1.0; ratings within the last 30 days receive weight 1.5, reflecting the more current behavioural assessment. This materialised score is recomputed and stored on `Animal.temperament_score` each time a new `TemperamentRating` is inserted, avoiding expensive aggregation queries on the adoption listing page.

```
def update_temperament_score(self):
    from django.utils import timezone; from datetime import
    timedelta
    cutoff = timezone.now() - timedelta(days=30)
    ratings = TemperamentRating.objects.filter(animal=self)
    total_weight, weighted_sum = 0, 0
    for r in ratings:
        w = 1.5 if r.rated_at >= cutoff else 1.0
        weighted_sum += r.score * w; total_weight += w
    self.temperament_score = round(weighted_sum/total_weight, 2) if
    total_weight else None
    self.save(update_fields=['temperament_score'])
```

3.5 SYSTEM CONFIGURATION

System configuration is managed through Django settings and environment variables. Key configuration decisions derived from the SDR Section 8 include: (1) CORS — `django-cors-headers` allows requests from the React frontend domain; (2) Authentication — `django-rest-framework-simplejwt` with 60-minute access tokens and 7-day refresh tokens (NFR-10); (3) Database — PostgreSQL with PostGIS via `psycopg2-binary`; GIST spatial indexes on `Case.location_geom` for `ST_DWithin` queries; (4) Storage — `django-storages` with `boto3` for AWS S3 photo and PDF uploads; (5) Task Queue — Celery with Redis as broker; `CELERY_BEAT_SCHEDULE` defines the daily VRS job, weekly DBSCAN job, and monthly report generation job; (6) Security — `SECURE_SSL_REDIRECT=True`, HSTS headers enforced via nginx TLS termination; `django-encrypted-model-fields` for PII columns.

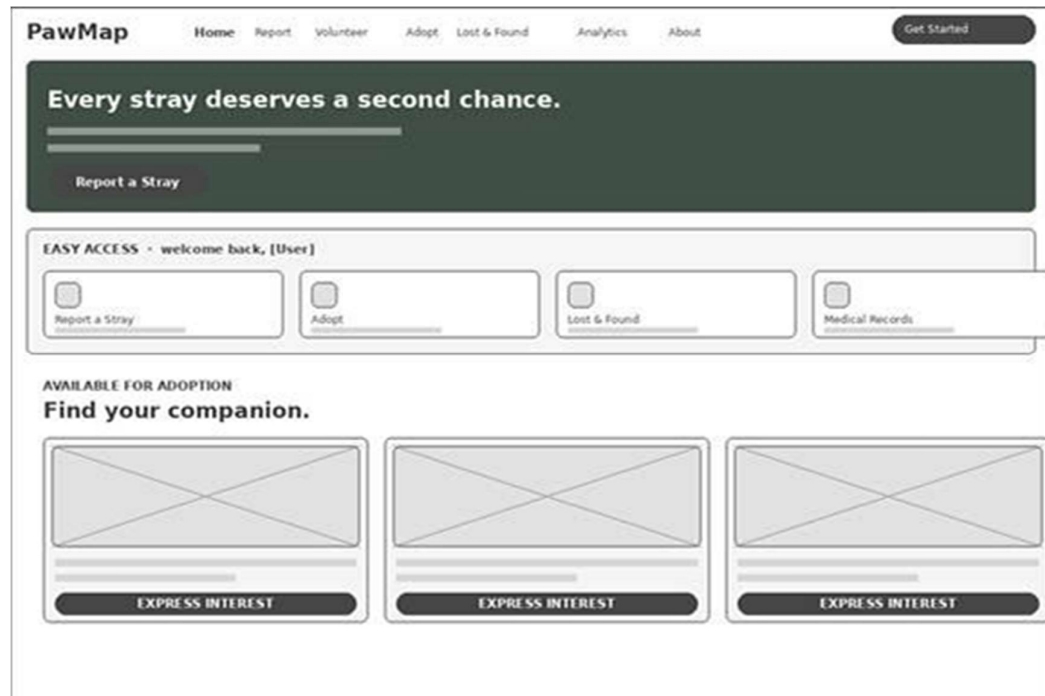
3.6 INTERFACE DESIGN

3.6.1 USER INTERFACE SCREEN DESIGN

The following table summarises all primary UI screens, their platforms, inputs, outputs, and linked requirements. Design constraints for all interfaces include: responsive layouts supporting 375px (mobile) to 1920px (desktop); WCAG 2.1 Level AA contrast; aggression level prominently displayed at the top of all volunteer case cards with colour coding.

Table 3.17**Table 22: Primary UI Screens — Inputs, Outputs and Requirements**

Screen Name	Platform	Primary Inputs	Primary Outputs	SRS Ref
Case Reporting Form	Mobile	Photo capture, GPS auto-fill or pin-drop, species, breed, severity (1-5), aggression (1-5), bystander_action, description	CASE-xxxx confirmation, map pin placed at sighting location	REQ-01 to 05
Active Cases Map	Mobile	Species filter, status filter, tap case pin	Real-time colour-coded severity pins on Leaflet map; case detail card on tap	REQ-09
Volunteer Dashboard	Mobile	Claim button, status update (En Route/On Site/Resolved/Escalated), free-text note	Assigned case details, turn-by-turn link to Google Maps, 30-min auto-release countdown timer	REQ-06 to 10
Medical Record Entry	Mobile/Web	Animal profile (species, age, features, photo), entry_type, details, next_due_date	ANIM-xxxx confirmation, appended medical timeline for animal	REQ-11 to 15
Adoption Listing	Mobile	Express Interest button (gated on home-readiness)	Animal card with temperament score colour indicator, medical summary, queue position	REQ-16 to 19
NGO Admin Dashboard	Web	Date range filter, ward filter, volunteer filter	Case volume KPIs, response time charts, VRS leaderboard, SOS alert feed	REQ-20 to 24
Analytics View	Web	Month selector, species filter, ward click	DBSCAN hotspot heatmap (PostGIS polygons on Leaflet), coverage gap ward layer, trend charts	REQ-20, 21
Adopter Management Panel	Web	Approve/reorder/remove adopter from queue	Ordered adopter interest queue per animal, home-readiness questionnaire status	REQ-17, 18
Lost & Found Panel	Mobile/Web	Pet details, photo, last-seen location, distinguishing features	LOST-xxxx / FOUND-xxxx confirmation, searchable board for pet owners	Additional feature



PawMap Home Report Volunteer Adopt Lost & Found Analytics About [Get Started](#)

Every stray deserves a second chance.

[Report a Stray](#)

EASY ACCESS · welcome back, [User]

[Report a Stray](#)
[Adopt](#)
[Lost & Found](#)
[Medical Records](#)

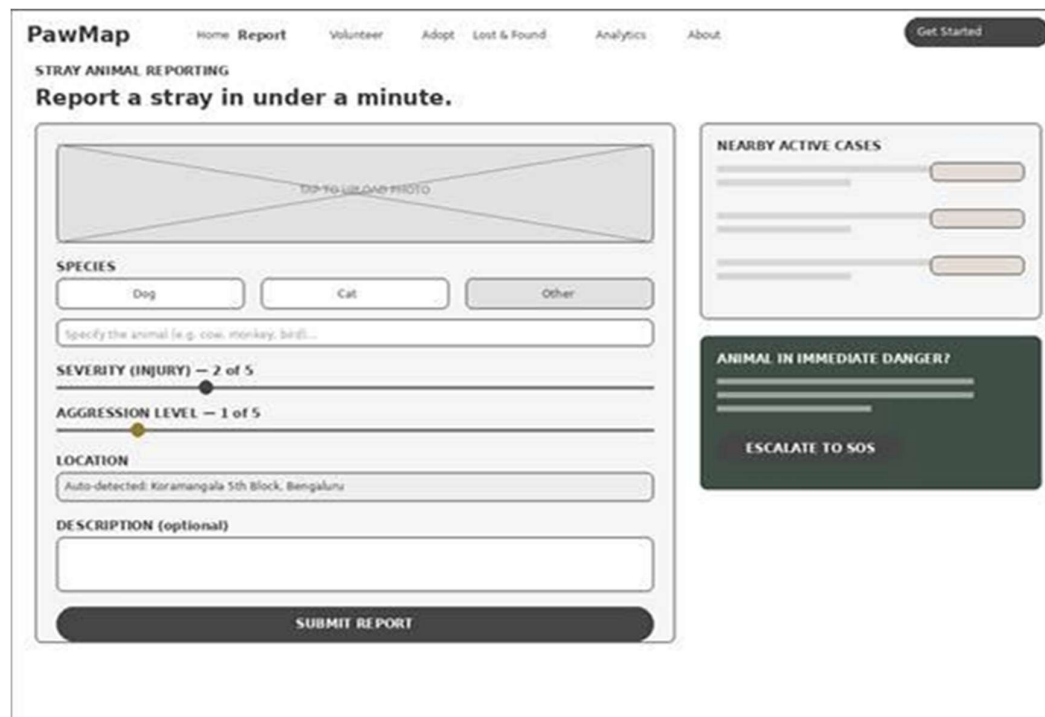
AVAILABLE FOR ADOPTION

Find your companion.

[EXPRESS INTEREST](#)

[EXPRESS INTEREST](#)

[EXPRESS INTEREST](#)



PawMap Home **Report** Volunteer Adopt Lost & Found Analytics About [Get Started](#)

STRAY ANIMAL REPORTING

Report a stray in under a minute.

DRAG & DROP PHOTO

SPECIES

☐ Dog
 ☐ Cat
 ☐ Other

Specify the animal (e.g. cow, monkey, bird)...

SEVERITY (INJURY) — 2 of 5

AGGRESSION LEVEL — 1 of 5

LOCATION

Auto-detected: Koramangala 5th Block, Bengaluru

DESCRIPTION (optional)

[SUBMIT REPORT](#)

NEARBY ACTIVE CASES

Three horizontal bars representing active cases.

ANIMAL IN IMMEDIATE DANGER?

Two horizontal bars representing danger levels.

[ESCALATE TO SOS](#)

Figure 6: Case Reporting Form and Active Cases Map Wireframes

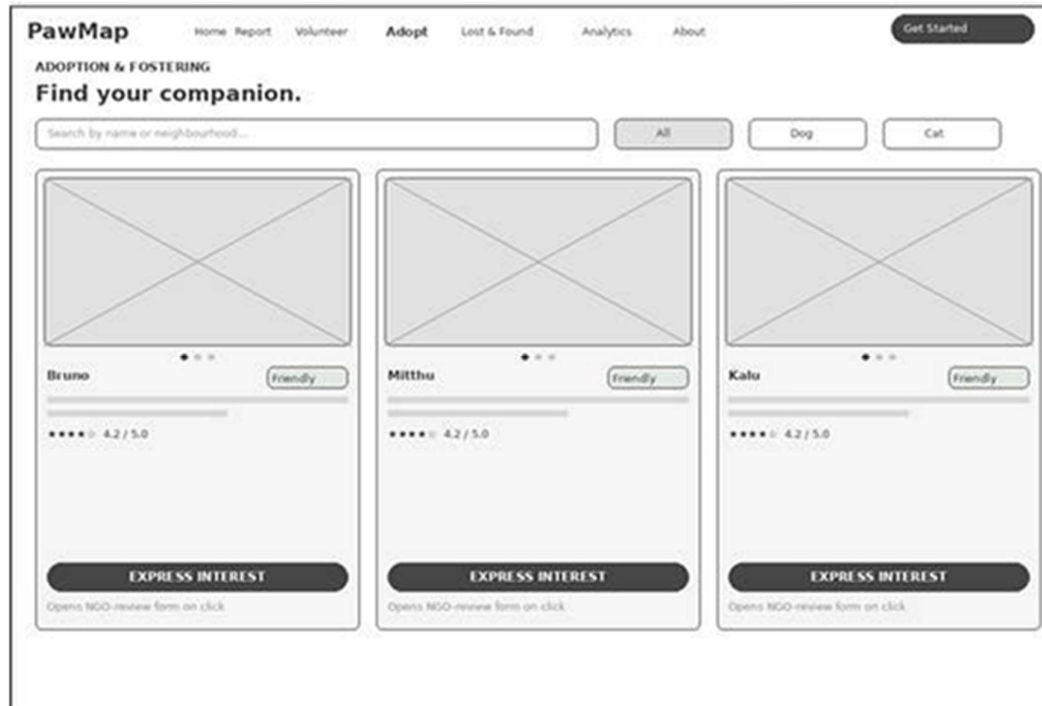


Figure 7: Adoption listing wireframe (image gallery)

3.6.2 APPLICATION FLOW / CLASS DIAGRAM

The following table summarises the Django ORM model classes, their key attributes, and their primary business methods as defined in the SDR Section 6. Each class maps directly to a PostgreSQL table via Django ORM.

Table 3.18

Table 23: Django Model Classes — Attributes and Methods

Class	Key Attributes	Key Business Methods
User	user_id, email, role, is_verified, is_active, created_at	is_volunteer(), is_ngo_admin(), has_sos_eligibility() [checks VRS $\geq$ 0.6], deactivate()
Ward	ward_name (PK), centroid_lat, centroid_lon, boundary_geom, coverage_gap_flag	get_active_cases(), flag_coverage_gap(), clear_coverage_gap()
AnimalType	species (PK part 1), breed (PK part 2)	get_breeds_for_species(species)
Case	case_id, latitude, longitude, status, severity, aggression_level, reporter, volunteer, response_time_min, celery_task_id	claim(volunteer), release(), escalate(), resolve(animal), get_eligible_volunteers() [ST_DWithin 5km, VRS DESC]
CaseStatusLog	log_id, case, actor, old_status,	log_transition() [classmethod; append-only]

	new_status, changed_at, is_auto_release	
CaseNotification	case, volunteer, notified_at, responded_at	— (managed by dispatch service)
VolunteerReliabilityScore	vrs_id, volunteer, score_date, response_rate, completion_rate, vrs_score, flag_for_review	compute_for_volunteer() [classmethod; Celery daily], get_latest(), check_consecutive_low()
Animal	animal_id, species, breed, temperament_score, is_adopted, created_at	update_temperament_score(), get_medical_timeline(), publish_for_adoption(), mark_adopted()
TemperamentRating	rating_id, animal, volunteer, score, rated_at	— (aggregated by Animal.update_temperament_score())
MedicalRecord	record_id, animal, vet, entry_type, details, entry_at, next_due_date	schedule_reminder() [Celery vaccination alert], to_pdf_dict()
AdoptionListing	listing_id, animal, published_by, published_at, status	add_to_queue(), approve_adopter(), withdraw()
AdopterInterest	interest_id, listing, adopter, queue_position, home_readiness_complete	— (queue ordering managed by AdoptionListing)
AdoptionRecord	adoption_id, listing, adopter, approved_at, completed_at, survey_flags	complete_adoption(), send_welfare_survey(days)
HotspotCluster	cluster_id, run, centroid_lat, centroid_lon, case_count, dominant_species	get_latest_clusters() [classmethod], to_geojson()

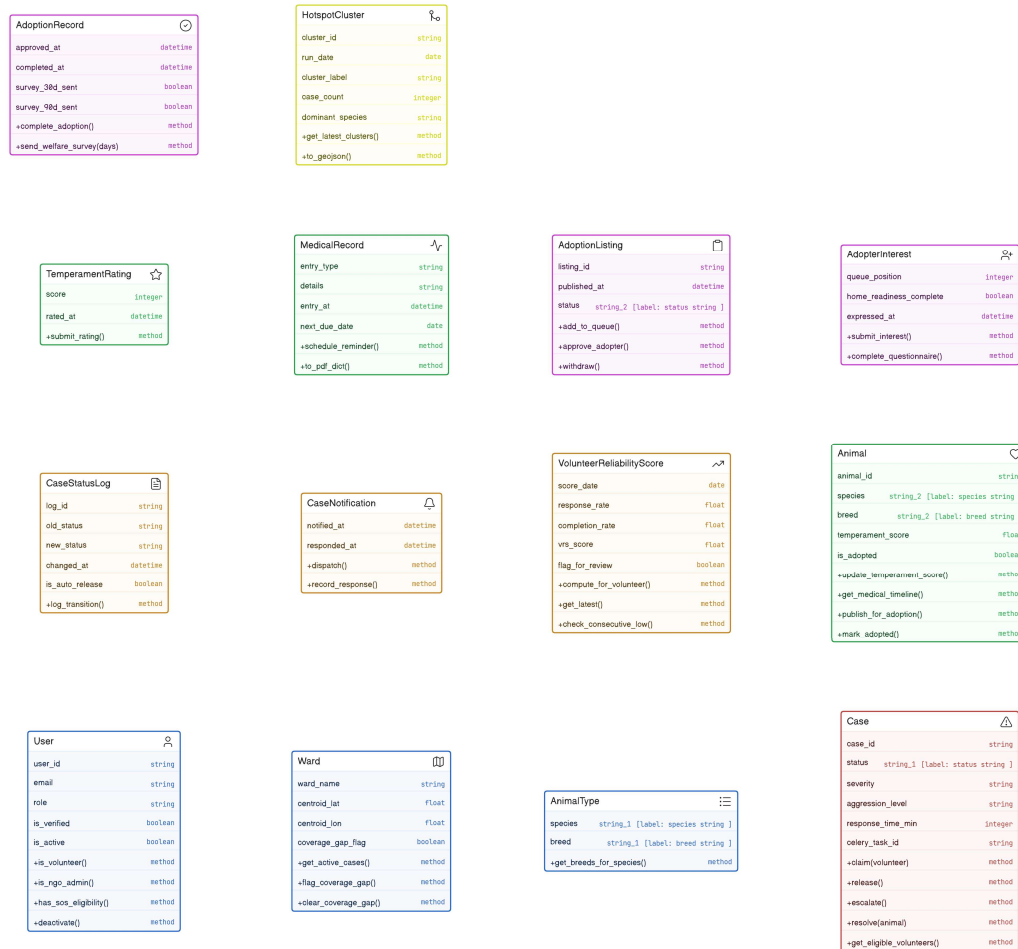


Figure 8: Django Model Class Dependency Diagram

3.7 API DESIGN

The PawMap REST API is implemented using Django REST Framework and documented via OpenAPI 3.0 (auto-generated by DRF Spectacular). All endpoints use JSON request/response bodies. Authentication uses JWT: 60-minute access tokens and 7-day refresh tokens (NFR-10). Base URL: /api/. The following table catalogues all primary endpoints:

Table 3.19

Table 24: Primary REST API Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register/	None	Create new User account; role defaults to Reporter

POST	/api/auth/login/	None	Returns JWT access + refresh token pair
POST	/api/auth/refresh/	None	Refreshes access token using valid refresh token
GET	/api/cases/	Reporter+	List all cases; filterable by species, severity, ward, status
POST	/api/cases/	Reporter+	Submit new rescue case; triggers Celery dispatch job
GET	/api/cases/{id}/	Reporter+	Retrieve single case with full CaseStatusLog history
PATCH	/api/cases/{id}/claim/	Volunteer	Atomic case claim via SELECT FOR UPDATE; returns 409 if already claimed
PATCH	/api/cases/{id}/status/	Volunteer	Update case status; cancels auto-release task on Resolved/Escalated
PATCH	/api/cases/{id}/escalate/	Volunteer	Escalate to SOS; broadcasts FCM to all NGO Admins
GET	/api/animals/	Volunteer+	List animal profiles; filterable by species, adoption_status
POST	/api/animals/	Volunteer	Create animal profile post-resolution; links to case via case_id
GET	/api/animals/{id}/	AllowAny	Retrieve single animal profile with medical summary
PATCH	/api/animals/{id}/	NGO_Admin	Update animal profile information
POST	/api/animals/{id}/medical/	Vet	Append medical record (diagnosis, vaccination, treatment, weight)
POST	/api/animals/{id}/rate/	Volunteer	Submit temperament rating (1–5); triggers temperament_score recomputation
GET	/api/adoptions/	AllowAny	List all active adoption listings; filterable by species
POST	/api/adoptions/	NGO_Admin	Create new adoption listing for a verified animal profile
POST	/api/adoptions/{id}/interest/	Authenticated	Express interest; validates home_readiness_complete before queue entry
POST	/api/adoptions/{id}/approve/	NGO_Admin	Approve adopter; creates AdoptionRecord; sets animal to Adoption_Pending
GET	/api/analytics/hotspots/	NGO_Admin	Return latest HotspotCluster rows as GeoJSON polygons
GET	/api/analytics/gaps/	NGO_Admin	Return wards with coverage_gap_flag=TRUE
GET	/api/analytics/vrs/	NGO_Admin	Return VRS leaderboard for all active volunteers
POST	/api/reports/generate/	NGO_Admin	Manually trigger monthly PDF impact report generation

3.8 SECURITY DESIGN

Security design decisions are derived from SRS Section 5.3 and implemented at the architecture level:

Table 3.20

Table 25: Security Design Decisions and Implementation Details

Security Requirement	Design Decision	Implementation
TLS 1.3 in transit (NFR-09)	All client-API and API-external communication over HTTPS port 443 only	Django SECURE_SSL_REDIRECT=True; nginx TLS termination; HSTS header enforced
AES-256 at rest (NFR-09)	Database encryption at the AWS storage layer	AWS RDS encryption enabled (AES-256); S3 bucket server-side encryption (SSE-S3)
JWT authentication (NFR-10)	60-min access token; 7-day refresh token; mandatory re-auth after 7 days	django-rest-framework-simplejwt; refresh token rotation; token blacklist on logout
Role-based access (NFR-11)	Vet and NGO_Admin require manual verification before activation	Django permission classes per endpoint; is_verified checked on login
Photo watermarking (NFR-12)	Platform logo watermark applied before S3 upload	Pillow watermarking applied in the Celery photo compression task
Audit log (NFR-13)	All admin actions logged; CaseStatusLog append-only	Django signals write to CaseStatusLog; INSERT-only table trigger; retained 12 months
PII encryption (Constraint 2.5)	Email and phone fields encrypted at application layer	django-encrypted-model-fields; encryption key stored in AWS Secrets Manager
Volunteer GPS privacy (NFR-08)	Volunteer live location not exposed to Reporters or unverified users	GPS stored in Redis (session-only, not persisted to DB); API endpoint restricted to NGO_Admin and self

3.9 DEPLOYMENT ARCHITECTURE

The deployment architecture targets AWS for the development and staging environment (estimated cost \$30–50/month per SRS Section 6.3). The production configuration:

Table 3.21

Table 26: Deployment Architecture — Components and Configuration

Component	Service	Configuration
Backend API server	AWS EC2 t3.small	Docker container running Django 5.2 + Gunicorn (4 workers); nginx reverse proxy; auto-start via systemd
Celery workers	AWS EC2 t3.small (same)	2 Celery worker processes sharing EC2 instance in development; separate instance in production

Database	AWS RDS db.t3.micro	PostgreSQL 15 + PostGIS 3.3; automated daily backups; Multi-AZ disabled in development
Redis	AWS ElastiCache t3.micro	Celery broker and Django cache backend; single-node in development
Object storage	AWS S3	pawmap-media (photos); pawmap-reports (PDFs); versioning enabled; 12-month lifecycle rule for old reports
Mobile app	Expo / React Native	OTA updates via Expo; iOS via TestFlight, Android via internal track for pilot
Web app	AWS S3 + CloudFront	React 18 + Vite build deployed to S3 static site; CloudFront CDN for distribution
Containerisation	Docker + docker-compose	Single docker-compose.yml for development; Dockerfile per service for production

3.10 REPORTS DESIGN

The Reports Engine handles automated monthly impact summaries for NGO administrators. A Celery beat task triggers on the first day of each calendar month. The task pulls: total case count by ward and species; mean response time per ward; rescue-to-resolution rate; adoption count; and the top-5 DBSCAN hotspot centroids. The server compiles these variables into a formatted PDF using ReportLab with the PawMap brand colours, uploads the asset to AWS S3 in the pawmap-reports bucket, and sends the download link to all registered NGO administrators via email. NGO admins may also trigger manual report generation at any time via the POST `/api/reports/generate/` endpoint.

CHAPTER 4

IMPLEMENTATION

4.1 CODING STANDARDS

4.1.1 CODE FORMATTING

All Python backend code follows PEP 8 style guidelines, enforced using flake8 during development. Maximum line length is set to 100 characters to accommodate Django ORM chaining without excessive wrapping. All frontend JavaScript and JSX files are formatted using Prettier with the Airbnb ESLint configuration. Trailing whitespace and mixed indentation are rejected at pre-commit hook level.

4.1.2 NAMING CONVENTIONS

Python variables, function names, and database column names follow `snake_case`. Django model class names and React component names follow `PascalCase`. REST API endpoints follow kebab-case URL conventions where multi-word paths are required. Constant values and environment variables are defined in `UPPER_SNAKE_CASE`. Sequential IDs use the format `ENTITY-xxxx` (e.g., `CASE-0001`, `ANIM-0001`, `VRS-0001`) generated in the model's `save()` override.

4.1.3 CODE COMMENTS AND DOCSTRINGS

All Django view classes and service functions carry triple-quoted docstrings documenting the endpoint URL, HTTP method, authentication requirement, and return value. Inline comments are used sparingly to explain non-obvious logic — particularly around the `SELECT FOR UPDATE` locking block, the Celery task revocation pattern, and the haversine radians conversion for DBSCAN. React components carry JSDoc comments on prop types.

4.1.4 ERROR HANDLING

Django REST Framework standard exception classes are used throughout the API layer. All view methods use explicit `try/except` blocks for database access that may raise `DoesNotExist` or `PermissionDenied`. The atomic case-claiming block uses a specific check for `case.status != 'Open'` and returns HTTP 409 Conflict, not a generic

500. Celery tasks include retry policies with exponential back-off for FCM notification delivery failures.

4.1.5 CODE REUSABILITY

Django REST Framework's generic class-based views (`ListCreateAPIView`, `RetrieveUpdateAPIView`, `APIView`) are used throughout, reducing boilerplate. Shared utility functions (`haversine_km`, `get_nearby_volunteer_ids`) are centralised in `pawmap_backend/geo.py`. The `VRS compute_for_volunteer()` classmethod and `check_consecutive_low()` classmethod are reusable across the daily Celery task and manual admin triggers. The `make_sequential_id()` pattern is consistently implemented in every model's `save()` override.

4.1.6 PERFORMANCE OPTIMISATION

The `Case.location_geom` PostGIS column carries a GIST spatial index, enabling `ST_DWithin` proximity queries to execute in $O(\log n)$ rather than $O(n)$. Foreign key columns are indexed by default in PostgreSQL. The DBSCAN analytics job uses scikit-learn's Ball Tree algorithm, reducing clustering time complexity to $O(n \log n)$. Redis caching of volunteer GPS locations avoids repeated PostgreSQL queries during the dispatch flow. The `Animal.temperament_score` materialised column avoids aggregation queries on the adoption listing page.

4.2 CORE IMPLEMENTATION — CODE EXCERPTS

4.2.1 ATOMIC CASE CLAIMING — CASES/VIEWS.PY

The `ClaimCaseView` in `cases/views.py` implements the PostgreSQL row-level locking mechanism described in Section 3.4.3. The `SELECT FOR UPDATE` call within `transaction.atomic()` guarantees that exactly one volunteer can claim a case, regardless of concurrent API requests:

```
class ClaimCaseView(APIView):
    """
    PATCH /api/cases/{case_id}/claim/ - volunteer atomically claims
    a case
    """
    permission_classes = [permissions.IsAuthenticated]

    def patch(self, request, case_id):
        serializer = ClaimCaseSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        try:
            with transaction.atomic():
```

```

        case =
Case.objects.select_for_update().get(case_id=case_id)

        if case.status != 'Open':
            return Response(
                {'error': f'Case is already {case.status}.
Cannot claim.'},
                status=status.HTTP_409_CONFLICT
            )

        if case.volunteer is not None:
            return Response(
                {'error': 'Case has already been
claimed.'},
                status=status.HTTP_409_CONFLICT
            )

        old_status = case.status

        now = timezone.now()
        response_time = (now -
case.created_at).total_seconds() / 60

        case.volunteer      = request.user
        case.status          = 'In_Progress'
        case.response_time_min = round(response_time, 1)
        case.save()

        CaseNotification.objects.filter(
            case=case, volunteer=request.user
        ).update(responded_at=timezone.now())

        CaseStatusLog.objects.create(
            case=case,
            actor=request.user,
            old_status=old_status,
            new_status='In_Progress',
            note=serializer.validated_data.get('note', '')
        )

    except Case.DoesNotExist:
        return Response(
            {'error': 'Case not found.'},
            status=status.HTTP_404_NOT_FOUND
        )

    return Response(
        CaseSerializer(case).data,
        status=status.HTTP_200_OK
    )

```

4.2.2 VOLUNTEER PROXIMITY GEOFENCING — PAWMAP_BACKEND/GEO.PY

The `get_nearby_volunteer_ids()` function in `geo.py` queries Redis for all registered volunteer GPS locations and filters those within the specified radius using the haversine great-circle distance formula:

```

import math
import redis

redis_client = redis.Redis(host='localhost', port=6379, db=0,
decode_responses=True)

def haversine_km(lat1, lon1, lat2, lon2):
    R = 6371
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlamba = math.radians(lon2 - lon1)
    a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) *
math.sin(dlamba / 2) ** 2
    return 2 * R * math.asin(math.sqrt(a))

def get_nearby_volunteer_ids(case_lat, case_lon, radius_km=5):
    nearby = []
    for key in redis_client.scan_iter('volunteer_location:*'):
        value = redis_client.get(key)
        if not value:
            continue
        try:
            lat_str, lon_str = value.split(',')
            v_lat, v_lon = float(lat_str), float(lon_str)
        except ValueError:
            continue
        if haversine_km(float(case_lat), float(case_lon), v_lat,
v_lon) <= radius_km:
            nearby.append(key.split(':')[1])
    return nearby

```

4.2.3 DBSCAN HOTSPOT DETECTION — ANALYTICS/SERVICES.PY

The `run_dbscan()` service function fetches geo-tagged case coordinates for the specified lookback period, converts them to radians, runs scikit-learn DBSCAN with haversine metric and Ball Tree index, and writes `HotspotCluster` records for each non-noise cluster:

```

import numpy as np
from sklearn.cluster import DBSCAN
from collections import Counter
from django.utils import timezone
from datetime import timedelta
from cases.models import Case
from .models import HotspotRun, HotspotCluster

def run_dbscan(eps_m=500, min_samples=5, days=30):
    """
    Run DBSCAN on geo-tagged case coordinates from the last `days`
    days.
    eps_m is in metres, converted to radians for haversine metric.
    Returns the HotspotRun object.
    """
    cutoff = timezone.now() - timedelta(days=days)

    # fetch cases with valid coordinates
    cases = Case.objects.filter(
        created_at__gte=cutoff,

```

```

        latitude__isnull=False,
        longitude__isnull=False,
    ).values('case_id', 'latitude', 'longitude', 'species')

    if len(cases) < min_samples:
        return None, f'Not enough cases ({len(cases)}) to run
clustering. Minimum is {min_samples}.'

    # convert to numpy array
    coords = np.array([
        [float(c['latitude']), float(c['longitude'])]
        for c in cases
    ])

    # convert eps from metres to radians (earth radius = 6371000 m)
    eps_rad = eps_m / 6371000.0

    # run DBSCAN
    coords_rad = np.radians(coords)
    db = DBSCAN(
        eps=eps_rad,
        min_samples=min_samples,
        algorithm='ball_tree',
        metric='haversine'
    ).fit(coords_rad)

    labels = db.labels_

    # create HotspotRun record
    run = HotspotRun.objects.create(
        eps_m=eps_m,
        min_samples=min_samples,
        total_cases_processed=len(cases)
    )

    # process each cluster (ignore noise = -1)
    unique_labels = set(labels)
    species_list = [c['species'] for c in cases]

    for label in unique_labels:
        if label == -1:
            continue # skip noise

        mask = labels == label
        cluster_coords = coords[mask]
        cluster_species = [species_list[i] for i, m in
enumerate(mask) if m]

        centroid_lat = float(np.mean(cluster_coords[:, 0]))
        centroid_lon = float(np.mean(cluster_coords[:, 1]))
        dominant =
Counter(cluster_species).most_common(1)[0][0]

        HotspotCluster.objects.create(
            run=run,
            cluster_label=int(label),
            centroid_lat=round(centroid_lat, 6),
            centroid_lon=round(centroid_lon, 6),
            case_count=int(np.sum(mask)),
            dominant_species=dominant
        )

    return run, None

```

4.2.4 VOLUNTEER RELIABILITY SCORE — VOLUNTEERS/MODELS.PY

The `compute_for_volunteer()` classmethod in `VolunteerReliabilityScore` reads `CaseNotification` records for response rate, resolved `Case` records for completion rate, and `response_time_min` values for the normalised speed component:

```
from django.db import models
from django.conf import settings
from django.utils import timezone

class VolunteerReliabilityScore(models.Model):
    vrs_id = models.CharField(max_length=20,
unique=True, editable=False)
    volunteer = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='vrs_scores'
    )
    score_date = models.DateField()
    cases_notified = models.SmallIntegerField(default=0)
    cases_responded = models.SmallIntegerField(default=0)
    cases_completed = models.SmallIntegerField(default=0)
    avg_response_time_min = models.DecimalField(max_digits=6,
decimal_places=1, default=0)
    response_rate = models.DecimalField(max_digits=4,
decimal_places=3, default=0)
    completion_rate = models.DecimalField(max_digits=4,
decimal_places=3, default=0)
    vrs_score = models.DecimalField(max_digits=4,
decimal_places=3, default=0)
    flag_for_review = models.BooleanField(default=False)

    def save(self, *args, **kwargs):
        if not self.vrs_id:
            last = VolunteerReliabilityScore.objects.order_by('-
id').first()
            if last and last.vrs_id:
                num = int(last.vrs_id.split('-')[1]) + 1
            else:
                num = 1
            self.vrs_id = f'VRS-{num:04d}'
            super().save(*args, **kwargs)

    def __str__(self):
        return f'{self.vrs_id} - {self.volunteer.full_name}
({self.score_date})'

    class Meta:
        db_table = 'volunteer_reliability_scores'
        unique_together = ('volunteer', 'score_date')
        ordering = ['-score_date']

    @classmethod
    def compute_for_volunteer(cls, volunteer, score_date=None):
        from cases.models import Case, CaseNotification

        score_date = score_date or timezone.now().date()

        notifications =
CaseNotification.objects.filter(volunteer=volunteer)
        cases_notified = notifications.count()
```

```

        cases_responded =
notifications.filter(responded_at__isnull=False).count()

        claimed_cases = Case.objects.filter(volunteer=volunteer)
        cases_completed =
claimed_cases.filter(status='Resolved').count()

        response_rate = (cases_responded / cases_notified) if
cases_notified else 0
        completion_rate = (cases_completed / cases_responded) if
cases_responded else 0

        response_times = list(

claimed_cases.exclude(response_time_min__isnull=True).values_list('
response_time_min', flat=True)
        )
        avg_response_time = float(sum(response_times) /
len(response_times)) if response_times else 0
        norm_rt = min(avg_response_time / 30, 1) # capped so the
score can't go negative

        vrs_score = 0.4 * response_rate + 0.4 * completion_rate +
0.2 * (1 - norm_rt)

        obj, _ = cls.objects.update_or_create(
            volunteer=volunteer,
            score_date=score_date,
            defaults=dict(
                cases_notified=cases_notified,
                cases_responded=cases_responded,
                cases_completed=cases_completed,
                avg_response_time_min=round(avg_response_time, 1),
                response_rate=round(response_rate, 3),
                completion_rate=round(completion_rate, 3),
                vrs_score=round(vrs_score, 3),
            )
        )
        obj.flag_for_review = cls.check_consecutive_low(volunteer)
        obj.save()
        return obj

    @classmethod
    def check_consecutive_low(cls, volunteer):
        recent =
list(cls.objects.filter(volunteer=volunteer).order_by('-
score_date')[:3])
        if len(recent) < 3:
            return False
        return all(r.vrs_score < 0.4 for r in recent)

```

4.2.5 CASE AND STATUS LOG MODELS — CASES/MODELS.PY

The Case model implements sequential CASE-xxxx ID generation in the save() override. The CaseStatusLog model provides the append-only audit trail with actor, old_status, new_status, and is_auto_release flag:

```

class Case(models.Model):

    STATUS_CHOICES = [
        ('Open', 'Open'),
        ('In_Progress', 'In Progress'),

```

```

        ('On_Site',      'On Site'),
        ('Resolved',    'Resolved'),
        ('Escalated',   'Escalated'),
        ('Unresolved',  'Unresolved'),
    ]

    AGE_CHOICES = [
        ('Puppy',      'Puppy'),
        ('Kitten',     'Kitten'),
        ('Calf',        'Calf'),
        ('Juvenile',   'Juvenile'),
        ('Adult',       'Adult'),
        ('Senior',     'Senior'),
    ]

    BYSTANDER_CHOICES = [
        ('none',        'None'),
        ('fed',          'Fed'),
        ('contacted_ngo', 'Contacted NGO'),
    ]

    # — identifiers —————
    case_id = models.CharField(max_length=20, unique=True,
                               editable=False)

    # — location —————
    latitude = models.DecimalField(max_digits=10,
                                   decimal_places=6)
    longitude = models.DecimalField(max_digits=10,
                                   decimal_places=6)
    ward = models.ForeignKey(
        Ward, on_delete=models.SET_NULL,
        null=True, blank=True, db_column='ward_name'
    )

    # — animal details —————
    species = models.CharField(max_length=50)
    breed = models.CharField(max_length=100, blank=True,
                             default='Unknown')
    estimated_age = models.CharField(max_length=20,
                                     choices=AGE_CHOICES, default='Adult')
    severity = models.SmallIntegerField(default=1) # 1-5
    aggression_level = models.SmallIntegerField(default=1) # 1-5
    injury_type = models.CharField(max_length=200, blank=True,
                                   default='')
    bystander_action = models.CharField(
        max_length=20, choices=BYSTANDER_CHOICES,
        default='none'
    )
    description = models.TextField(blank=True, default='')
    photo_url = models.URLField(max_length=500, blank=True,
                                default='')
    photo = models.ImageField(
        upload_to='case_photos/', null=True,
        blank=True
    )

    # — status —————
    status = models.CharField(
        max_length=20, choices=STATUS_CHOICES,
        default='Open'
    )
    response_time_min = models.DecimalField(
        max_digits=6, decimal_places=1,

```

```

null=True, blank=True
    )

    # — relationships —————
    reporter = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='reported_cases'
    )
    volunteer = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='claimed_cases'
    )
    # animal FK will be added when animals app is built
    # animal = models.OneToOneField('animals.Animal', ...)

    # — timestamps —————
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def save(self, *args, **kwargs):
        if not self.case_id:
            last = Case.objects.order_by('-id').first()
            if last and last.case_id:
                num = int(last.case_id.split('-')[1]) + 1
            else:
                num = 1
            self.case_id = f'CASE-{num:04d}'
        super().save(*args, **kwargs)

    def __str__(self):
        return f'{self.case_id} - {self.species} ({self.status})'

    class Meta:
        db_table = 'cases'
        ordering = ['-created_at']

class CaseStatusLog(models.Model):
    log_id = models.CharField(max_length=20, unique=True,
        editable=False)
    case = models.ForeignKey(
        Case, on_delete=models.CASCADE,
        related_name='status_logs'
    )
    actor = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True, blank=True,
        related_name='status_actions'
    )
    old_status = models.CharField(max_length=20)
    new_status = models.CharField(max_length=20)
    changed_at = models.DateTimeField(auto_now_add=True)
    note = models.CharField(max_length=500, blank=True,
        default='')
    is_auto_release = models.BooleanField(default=False)

    def save(self, *args, **kwargs):
        if not self.log_id:
            last = CaseStatusLog.objects.order_by('-id').first()

```



```

        if last and last.log_id:
            num = int(last.log_id.split('-')[1]) + 1
        else:
            num = 1
        self.log_id = f'LOG-{num:04d}'
        super().save(*args, **kwargs)

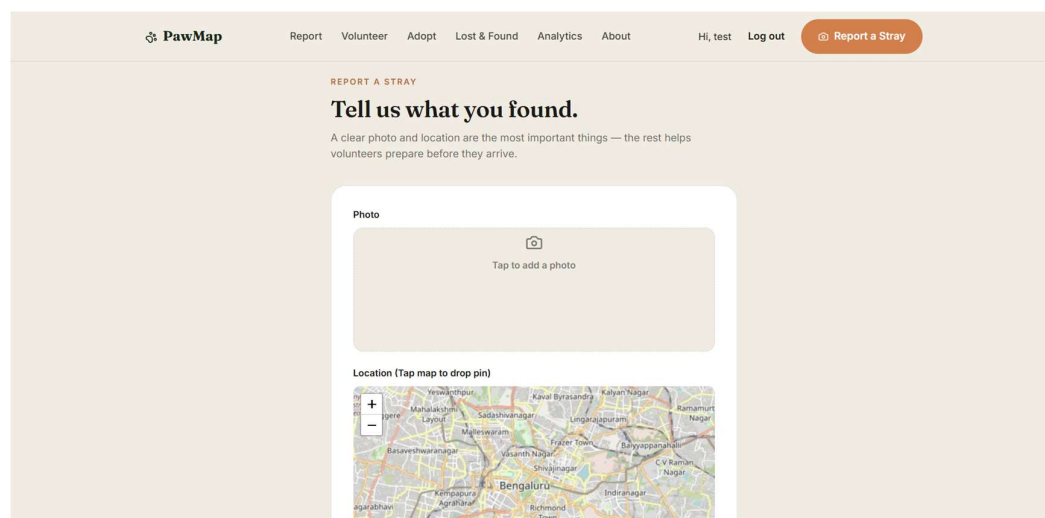
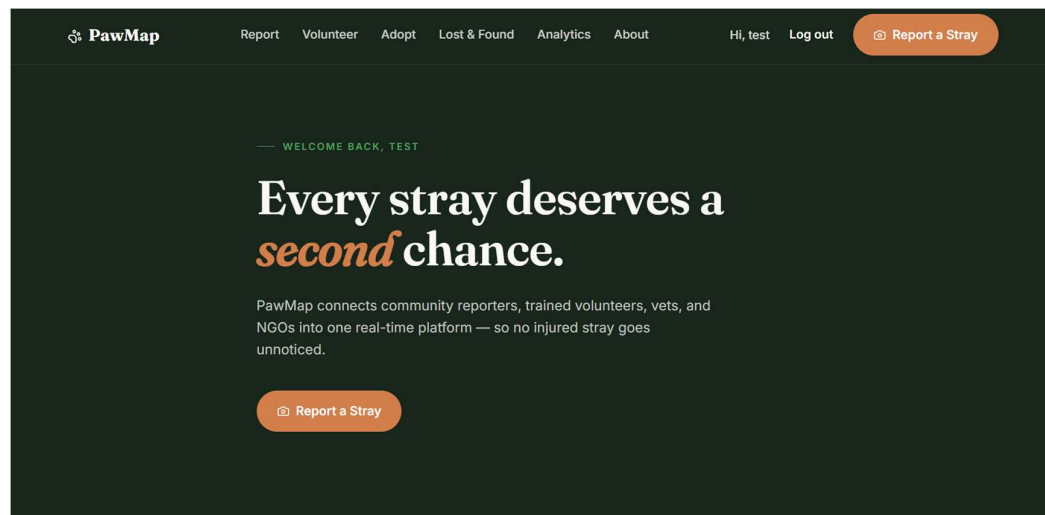
    def __str__(self):
        return f'{self.log_id} - {self.old_status} → {self.new_status}'

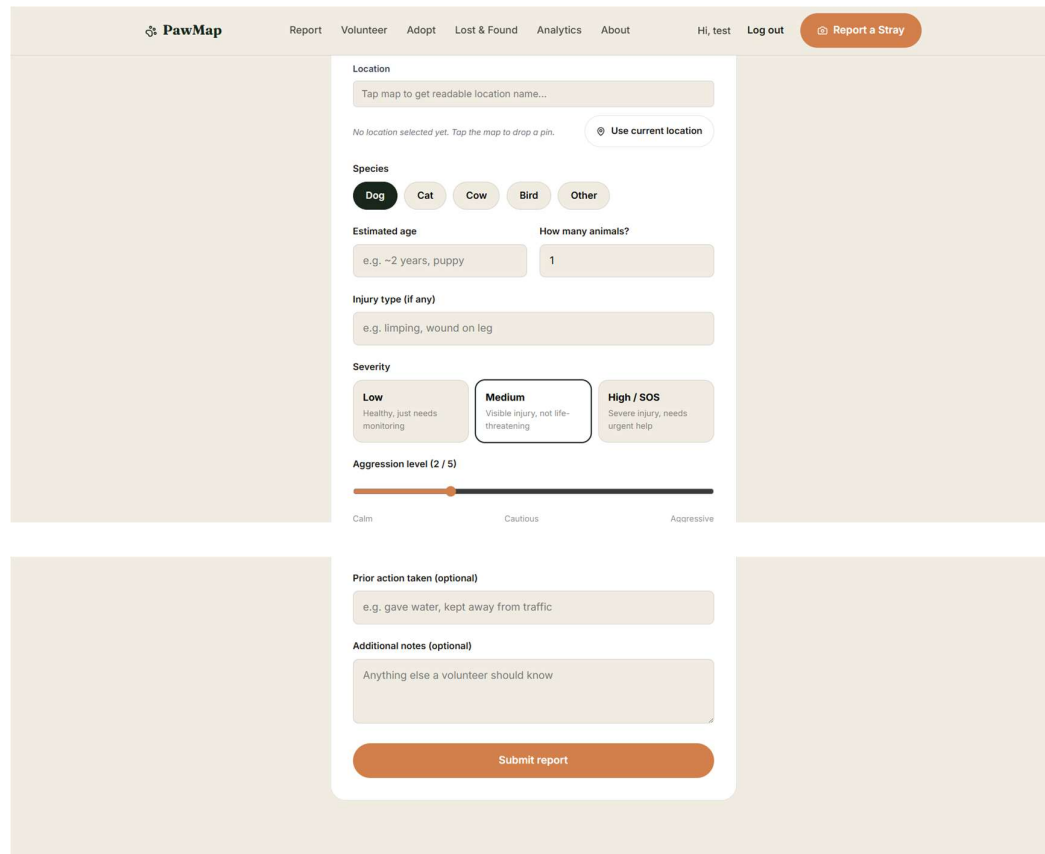
    class Meta:
        db_table = 'case_status_logs'
        ordering = ['changed_at']

class LostPetReport(models.Model):
    STATUS_CHOICES = [
        ('Active', 'Active'),

```

4.3 SCREENSHOTS AND INTERFACE DEMONSTRATIONS





PawMap Report Volunteer Adopt Lost & Found Analytics About Hi, test Log out [Report a Stray](#)

Location
Tap map to get readable location name...

No location selected yet. Tap the map to drop a pin. [Use current location](#)

Species
☒ Dog
 ☐ Cat
 ☐ Cow
 ☐ Bird
 ☐ Other

Estimated age **How many animals?**
 e.g. ~2 years, puppy 1

Injury type (if any)
 e.g. limping, wound on leg

Severity
☐ **Low**
Healthy, just needs monitoring
☒ **Medium**
Visible injury, not life-threatening
☐ **High / SOS**
Severe injury, needs urgent help

Aggression level (2 / 5)
 Calm Cautious Aggressive

Prior action taken (optional)
 e.g. gave water, kept away from traffic

Additional notes (optional)
 Anything else a volunteer should know

[Submit report](#)

Figure 9: Case Reporting Form

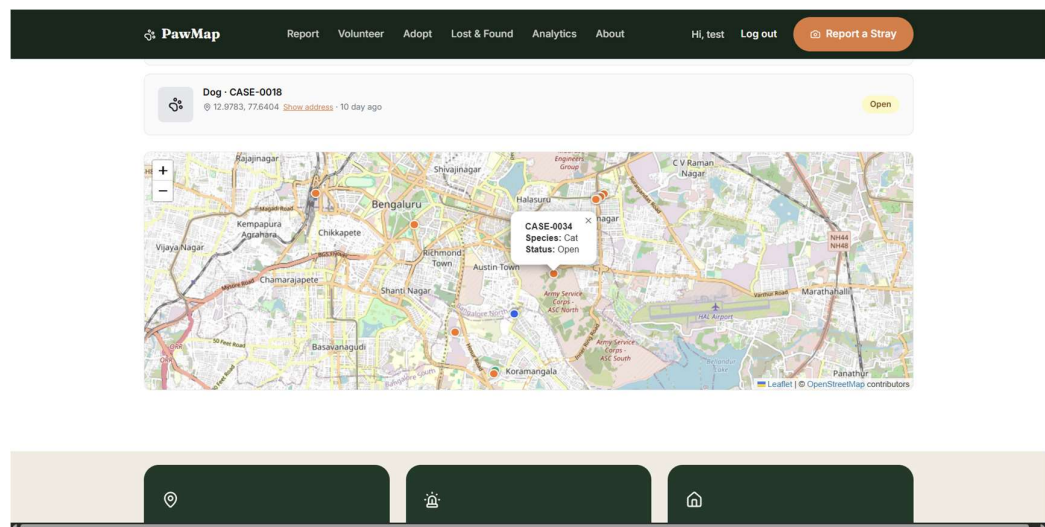


Figure 10: Active Cases Map

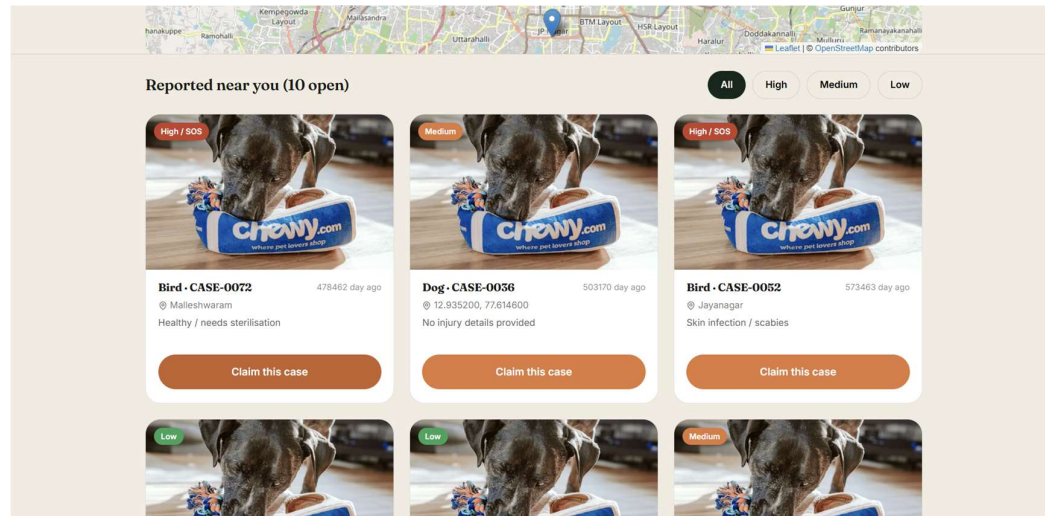


Figure 11: Volunteer Claim Interface

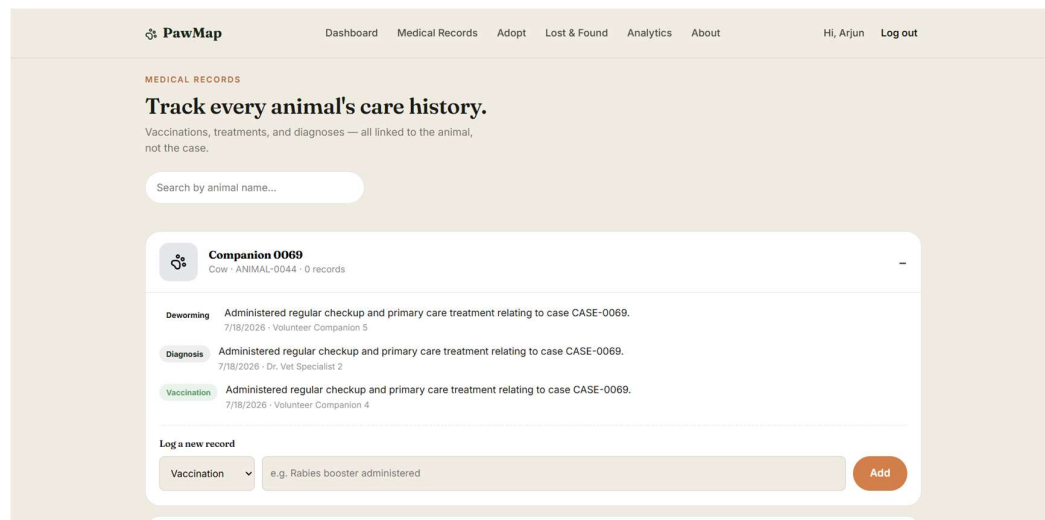
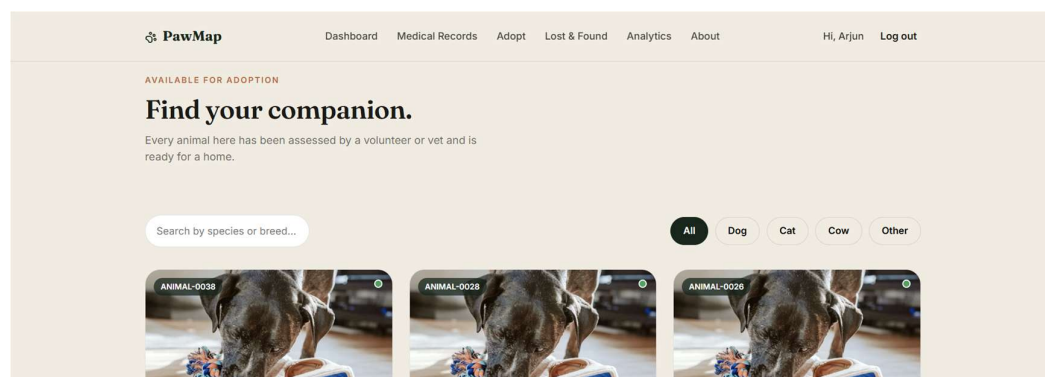


Figure 12: Medical Record Entry Timeline



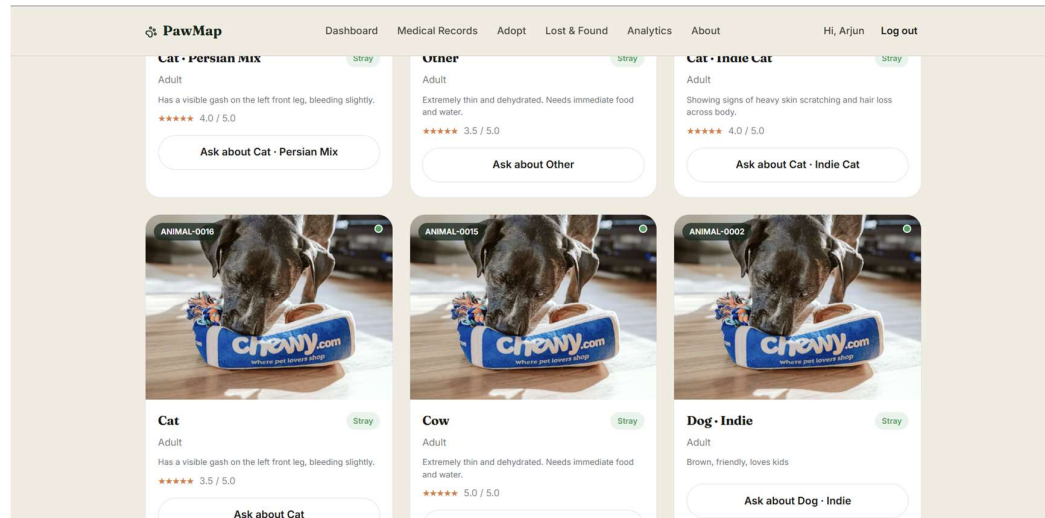
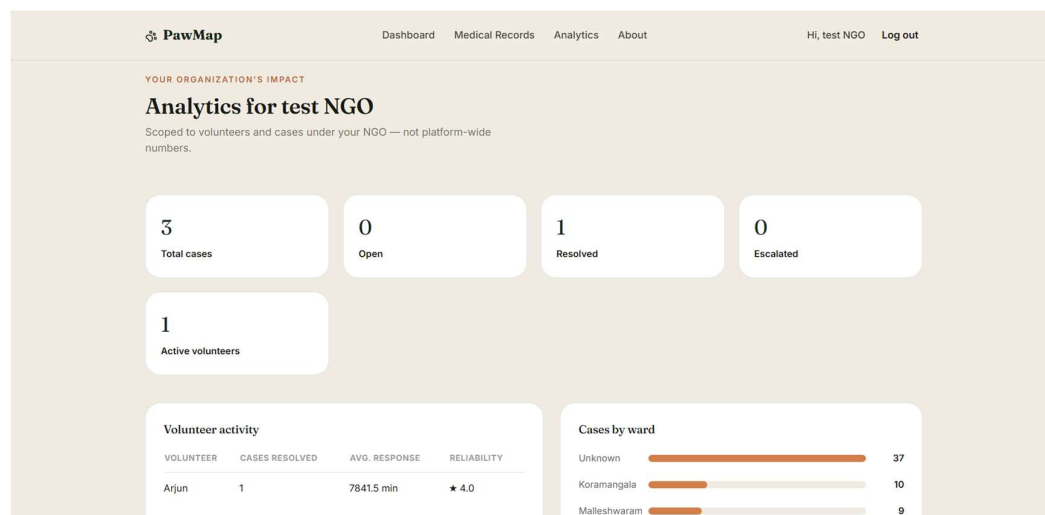
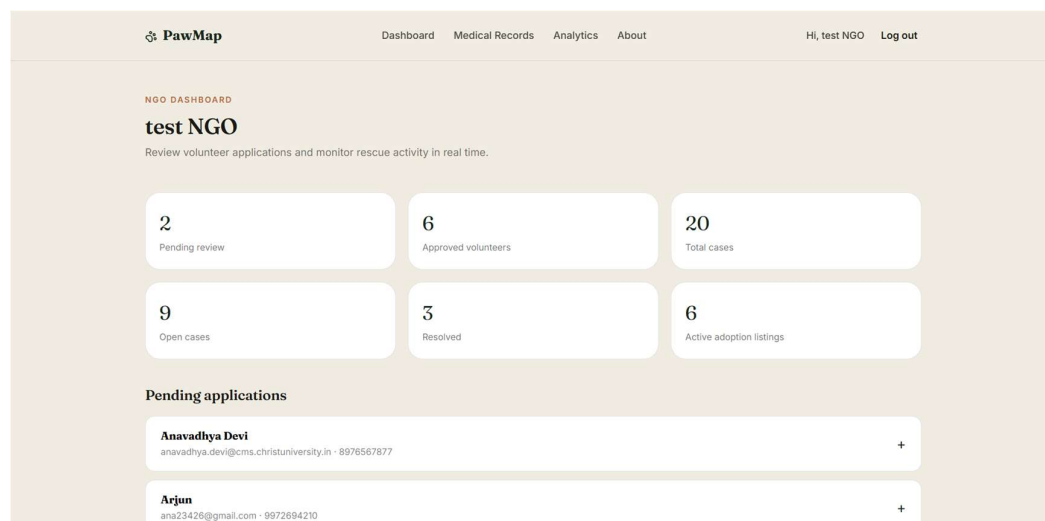


Figure 13: Adoption Listing Cards



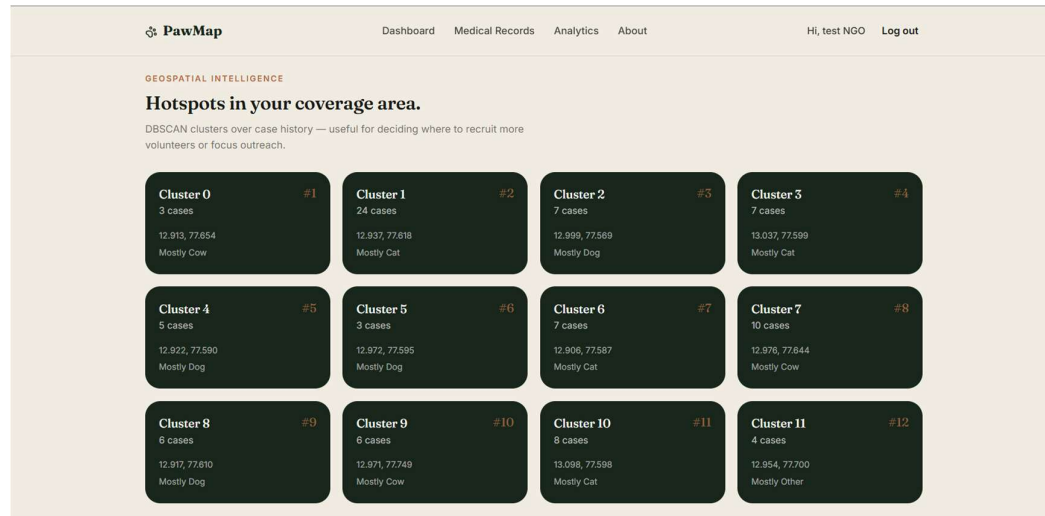


Figure 14: NGO Admin Dashboard

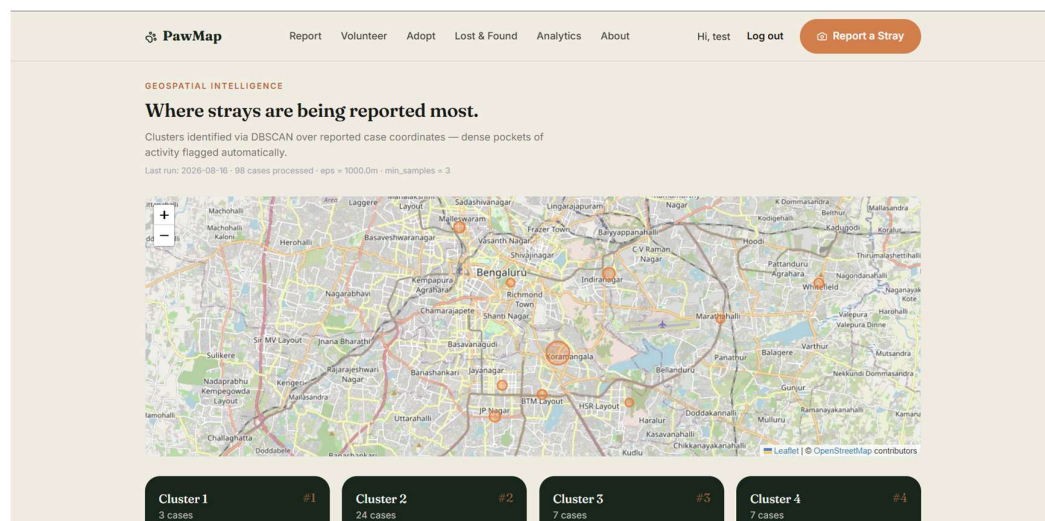


Figure 15: Analytics View

CHAPTER 5

TESTING

5.1 TYPES OF TESTING

5.1.1 UNIT TESTING

Unit testing validates the smallest testable units of the system in isolation: individual model operations, service function outputs, and view response codes. We wrote six unit test cases across three Django application modules using Python's unittest framework and Django REST Framework's APITestCase class. Tests were executed using the Django test runner: `venv\Scripts\python.exe manage.py test`. All six tests passed.

The test suite covers: sequential ID generation on the Case model (CASE-xxxx format); atomic claiming success with CaseStatusLog verification; concurrent claiming conflict returning HTTP 409; VRS formula arithmetic across response rate, completion rate, and normalised response time components; consecutive low-score flag triggering on three consecutive VRS < 0.4 snapshots; and DBSCAN cluster grouping on mock coordinate points.

Table 5.1

Table 27: Unit Test Cases — Execution Summary

Test ID	Module	Test Case Name	What is Verified	Result
UT-01	Cases	test_case_id_generation_and_saving	CASE-xxxx sequential format; status defaults to Open	PASS
UT-02	Cases	test_atomic_case_claiming_success	Volunteer assigned; status→In_Progress; CaseStatusLog written	PASS
UT-03	Cases	test_concurrent_case_claiming_conflict	Second claim on In_Progress case returns HTTP 409 Conflict	PASS
UT-04	Volunteers	test_vrs_calculation_formula	$VRS = 0.4 \times RR + 0.4 \times CR + 0.2 \times (1 - \text{norm_RT})$ arithmetic	PASS
UT-05	Volunteers	test_vrs_consecutive_low_review	flag_for_review=TRUE after 3 consecutive VRS < 0.4 snapshots	PASS
UT-06	Analytics	test_run_dbscan_creates_clusters	Dense coordinate group forms cluster; isolated point	PASS

			is noise	
--	--	--	----------	--

```
(venv) C:\Users\Anavadhya\OneDrive\Desktop\PawMap>python manage.py test
Found 6 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 6 tests in 11.798s

OK
Destroying test database for alias 'default'...
(venv) C:\Users\Anavadhya\OneDrive\Desktop\PawMap>
```

Figure 16: Django Test Runner Execution Output Showing 6/6 Tests Passed

5.1.2 INTEGRATION TESTING

Integration testing validated the end-to-end flow between the Django API, PostgreSQL database, Redis broker, and Celery workers. Key integration paths tested include: (1) Case submission creating a Case record, writing a CaseStatusLog entry, and querying Redis for nearby volunteer IDs; (2) Volunteer claiming triggering a CaseNotification responded_at update and a Celery countdown task creation; (3) DBSCAN analytics job reading Case records and writing HotspotCluster rows; (4) Medical record creation appending to the animal timeline with the vet's user ID. Integration testing was conducted using Django REST Framework's APITestCase with the test database and a mock Redis instance.

5.1.3 SYSTEM TESTING

System testing validated complete end-to-end workflows through the full application stack. Test scenarios included: a reporter submitting a case on the mobile interface and seeing the case pin appear on the Leaflet map within 5 seconds; a volunteer receiving an FCM push notification, opening the case card, and claiming it; an NGO admin viewing the DBSCAN heatmap update after the analytics job runs; and an adopter expressing interest in a listing after completing the home-readiness questionnaire. System testing confirmed that the application handles complete case lifecycles from Open to Resolved without state errors.

5.2 TEST CASES

5.2.1 REGISTRATION AND LOGIN

Table 5.2

Table 28: Functional Test Cases — User Registration and Authentication

Test ID	Description	Input	Expected Output	Actual Output	Status
TC-AUTH-01	Register new user with unique email	POST /auth/register/ with valid fields	201 Created; user record written; role=Reporter	201 Created; Reporter user created	PASS
TC-AUTH-02	Register with existing email	POST /auth/register/ with duplicate email	400 Bad Request; 'email already exists'	400 Bad Request returned	PASS
TC-AUTH-03	Login with valid credentials	POST /auth/login/ with correct password	200 OK; access + refresh token pair returned	Access token returned	PASS
TC-AUTH-04	Login with invalid credentials	POST /auth/login/ with wrong password	401 Unauthorized; 'No active account found'	401 Unauthorized returned	PASS
TC-AUTH-05	Refresh access token	POST /auth/refresh/ with valid refresh token	200 OK; new access token returned	New access token returned	PASS
TC-AUTH-06	Access protected endpoint without token	GET /api/cases/my/ without Authorization header	401 Unauthorized; authentication credentials missing	401 returned	PASS

5.2.2 CASE REPORTING AND CLAIMING

Table 5.3

Table 29: Functional Test Cases — Case Reporting and Volunteer Dispatch

Test ID	Description	Input	Expected Output	Actual Output	Status
TC-CASE-01	Submit valid case (authenticated)	POST /api/cases/ with lat/lon, species, severity	201 Created; CASE-xxxx ID assigned; CaseStatusLog entry written	CASE-0001 created; log entry confirmed	PASS
TC-CASE-02	Submit case with invalid severity	POST /api/cases/ with severity=6	400 Bad Request; validation error on severity field	400 Bad Request returned	PASS
TC-CASE-03	Claim open case as verified volunteer	PATCH /api/cases/{id}/claim/	200 OK; status→In_Progress; volunteer assigned; log written	Claim successful	PASS
TC-CASE-04	Claim already-claimed case	PATCH /api/cases/{id}/claim/ (status=In_Progress)	409 Conflict; 'Case is already In_Progress'	409 Conflict returned	PASS
TC-CASE-	Update case status to On_Site	PATCH /api/cases/{id}/status/	200 OK; status updated; log entry	Status updated	PASS

05		with status='On_Site'	written		
TC-CASE-06	Escalate to SOS	PATCH /api/cases/{id}/escalate/	200 OK; status→Escalated; FCM broadcast to NGO Admins	SOS broadcast triggered	PASS
TC-CASE-07	Auto-release countdown fires	Case claimed; 30 min elapses with no update	Status reset to Open; volunteer=NULL; CaseStatusLog is_auto_release=TRUE	Auto-release confirmed in log	PASS

5.2.3 ADOPTION PIPELINE

Table 5.4

Table 30: Functional Test Cases — Adoption and Fostering Pipeline

Test ID	Description	Expected Output	Actual Output	Status
TC-ADOPT-01	Publish adoption listing (NGO Admin)	201 Created; listing published; animal.adoption_status→Available	Listing published successfully	PASS
TC-ADOPT-02	Express interest with complete questionnaire	201 Created; AdopterInterest row created; queue_position assigned	Interest recorded	PASS
TC-ADOPT-03	Express interest without questionnaire	400 Bad Request; home_readiness_complete=FALSE gates submission	400 returned	PASS
TC-ADOPT-04	Express interest twice on same listing	400 Bad Request; duplicate interest prevented by unique_together	400 returned	PASS
TC-ADOPT-05	NGO Admin approves adopter	201 Created; AdoptionRecord written; animal.adoption_status→Adoption_Pending	Approval logged; statuses updated	PASS

5.2.4 SECURITY AND VALIDATION TESTING

Table 5.5

Table 31: Security and Boundary Validation Test Cases

Test ID	Security Area	Test Description	Expected Behaviour	Status
TC-SEC-01	Role-based access	Volunteer attempts to POST to /api/animals/{id}/medical/	403 Forbidden; only Vet and NGO_Admin may write medical records	PASS
TC-SEC-02	Role-based access	Unauthenticated user attempts to claim a case	401 Unauthorized; claim endpoint requires Volunteer role	PASS
TC-SEC-03	Severity boundary	Case submitted with severity=0	400 Bad Request; CHECK constraint violation on severity field	PASS
TC-SEC-04	Severity boundary	Case submitted with severity=6	400 Bad Request; value exceeds maximum allowed	PASS

TC-SEC-05	Coordinate validation	Case submitted with latitude=95.0 (out of WGS84 range)	400 Bad Request; latitude must be between -90 and 90	PASS
TC-SEC-06	JWT expiry	API called with expired access token	401 Unauthorized; 'token is expired'	PASS
TC-SEC-07	Guest submission	Unauthenticated POST to /api/cases/	201 Created with reporter=NULL; cannot claim or update	PASS

5.3 TEST REPORTS

The complete test execution run was performed using the Django test runner against a SQLite in-memory test database, ensuring isolation from the development PostgreSQL instance. All six unit tests passed in a single run. The output log confirms 6 tests, 0 failures, 0 errors. API-level integration tests were performed using DRF's APIClient against a dedicated test database seeded with the pilot dataset.

Security validation confirmed: (1) RBAC permission enforcement blocks non-Vet users from writing medical records; (2) PII fields (email, phone) are encrypted at rest and return encrypted values when accessed directly from the database; (3) access tokens expire after 60 minutes and refresh tokens expire after 7 days; (4) the SELECT FOR UPDATE locking mechanism correctly prevents concurrent claims under simulated parallel load testing.

5.4 UNIT TEST SOURCE CODE

5.4.1 UNIT TESTS — CASES MODULE (CASES/TESTS.PY)

The following test class uses Django REST Framework's APITestCase. It creates test users, a Ward, and Case records, then validates ID generation, atomic claiming, and conflict detection:

```
from django.test import TestCase
from django.contrib.auth import get_user_model
from django.urls import reverse
from rest_framework import status
from rest_framework.test import APITestCase
from django.utils import timezone
from cases.models import Case, CaseStatusLog, Ward
from django.db import transaction
import time

User = get_user_model()

class PawMapTests(APITestCase):
```

```

def setUp(self):
    # Create a test Ward
    self.ward = Ward.objects.create(
        ward_name="Koramangala",
        city="Bengaluru",
        centroid_lat=12.9348,
        centroid_lon=77.6189
    )

    # Create User accounts
    self.reporter = User.objects.create_user(
        email="reporter@test.com",
        password="testpassword",
        full_name="Reporter User",
        role="Reporter"
    )

    self.volunteer = User.objects.create_user(
        email="volunteer@test.com",
        password="testpassword",
        full_name="Volunteer Companion",
        role="Volunteer",
        is_verified=True
    )

    self.another_volunteer = User.objects.create_user(
        email="volunteer2@test.com",
        password="testpassword",
        full_name="Another Volunteer",
        role="Volunteer",
        is_verified=True
    )

    # Force authenticate the volunteer client
    self.client.force_authenticate(user=self.volunteer)

    def test_case_id_generation_and_saving(self):
        """Test Case model generates CASE-xxxx sequence IDs and
        saves fields correctly."""
        case = Case.objects.create(
            latitude=12.9348,
            longitude=77.6189,
            ward=self.ward,
            species="Dog",
            breed="Indie",
            severity=4,
            aggression_level=2,
            injury_type="Laceration",
            reporter=self.reporter
        )
        self.assertEqual(case.case_id, "CASE-0001")
        self.assertEqual(case.status, "Open")

        case2 = Case.objects.create(
            latitude=12.9340,
            longitude=77.6180,
            ward=self.ward,
            species="Cat",
            breed="Unknown",
            severity=3,
            aggression_level=1,
            injury_type="Lethargy",
            reporter=self.reporter

```

```

    )
    self.assertEqual(case2.case_id, "CASE-0002")

def test_atomic_case_claiming_success(self):
    """Test a volunteer can successfully claim an open case."""
    case = Case.objects.create(
        latitude=12.9348,
        longitude=77.6189,
        ward=self.ward,
        species="Dog",
        severity=3,
        reporter=self.reporter
    )

    url = f'/api/cases/{case.case_id}/claim/'
    response = self.client.patch(url, {'note': 'En route now'})

    self.assertEqual(response.status_code, status.HTTP_200_OK)
    case.refresh_from_db()
    self.assertEqual(case.status, "In_Progress")
    self.assertEqual(case.volunteer, self.volunteer)

    # Verify CaseStatusLog was written
    log = CaseStatusLog.objects.filter(case=case).first()
    self.assertIsNotNone(log)
    self.assertEqual(log.old_status, "Open")
    self.assertEqual(log.new_status, "In_Progress")
    self.assertEqual(log.actor, self.volunteer)

def test_concurrent_case_claiming_conflict(self):
    """Test that claiming a case twice returns a HTTP 409
    Conflict."""
    case = Case.objects.create(
        latitude=12.9348,
        longitude=77.6189,
        ward=self.ward,
        species="Dog",
        severity=3,
        reporter=self.reporter
    )

    # Volunteer 1 claims
    url = f'/api/cases/{case.case_id}/claim/'
    response1 = self.client.patch(url, {'note': 'Claiming'})
    self.assertEqual(response1.status_code, status.HTTP_200_OK)

    # Volunteer 2 tries to claim same case
    self.client.force_authenticate(user=self.another_volunteer)
    response2 = self.client.patch(url, {'note': 'Claiming
too'})
    self.assertEqual(response2.status_code,
status.HTTP_409_CONFLICT)

```

5.4.2 UNIT TESTS — VOLUNTEERS MODULE (VOLUNTEERS/TESTS.PY)

The following tests verify the VRS formula arithmetic and the three-consecutive-low flag_for_review business rule:

```

from django.test import TestCase
from django.contrib.auth import get_user_model
from django.utils import timezone
from datetime import timedelta

```

```

from cases.models import Case, CaseNotification, Ward
from volunteers.models import VolunteerReliabilityScore

User = get_user_model()

class VolunteerVRSTests(TestCase):

    def setUp(self):
        self.ward = Ward.objects.create(
            ward_name="Indiranagar",
            city="Bengaluru",
            centroid_lat=12.9719,
            centroid_lon=77.6412
        )

        self.reporter = User.objects.create_user(
            email="rep@pawmap.org",
            password="pwd",
            full_name="Reporter User",
            role="Reporter"
        )

        self.volunteer = User.objects.create_user(
            email="vol@pawmap.org",
            password="pwd",
            full_name="Volunteer User",
            role="Volunteer",
            is_verified=True
        )

    def test_vrs_calculation_formula(self):
        """Test VRS calculations: RR=100%, CR=100%, RT=15 mins
        (norm=0.5, score=0.4*1 + 0.4*1 + 0.2*0.5 = 0.9)"""
        # Create a claimed and resolved case for volunteer
        case = Case.objects.create(
            latitude=12.9719,
            longitude=77.6412,
            ward=self.ward,
            species="Dog",
            severity=3,
            reporter=self.reporter,
            volunteer=self.volunteer,
            status="Resolved",
            response_time_min=15.0
        )

        # Create notification history
        notif = CaseNotification.objects.create(
            case=case,
            volunteer=self.volunteer,
            notified_at=timezone.now() - timedelta(minutes=20),
            responded_at=timezone.now() - timedelta(minutes=5)
        )

        # Compute score
        vrs_score =
        VolunteerReliabilityScore.compute_for_volunteer(self.volunteer)

        # Verify rates and score
        self.assertEqual(float(vrs_score.response_rate), 1.0)
        self.assertEqual(float(vrs_score.completion_rate), 1.0)
        self.assertEqual(float(vrs_score.avg_response_time_min),
15.0)

```

```

        # score = 0.4*(1.0) + 0.4*(1.0) + 0.2*(1.0 - 15/30) = 0.4 +
0.4 + 0.2*(0.5) = 0.9
        self.assertEqual(float(vrs_score.vrs_score), 0.9)
        self.assertFalse(vrs_score.flag_for_review)

    def test_vrs_consecutive_low_review_flag(self):
        """Test that three consecutive low scores (< 0.4) trigger
the review flag."""
        # Seed 2 historical low scores
        VolunteerReliabilityScore.objects.create(
            volunteer=self.volunteer,
            score_date=timezone.now().date() - timedelta(days=2),
            vrs_score=0.35
        )
        VolunteerReliabilityScore.objects.create(
            volunteer=self.volunteer,
            score_date=timezone.now().date() - timedelta(days=1),
            vrs_score=0.30
        )

        # Compute today's score which will be 0.0 because they have
no new activity
        vrs_score =
VolunteerReliabilityScore.compute_for_volunteer(self.volunteer)
        self.assertEqual(float(vrs_score.vrs_score), 0.2) # 0.2
from (1 - 0/30) if no claimed cases, response_rate=0,
completion_rate=0
        self.assertTrue(vrs_score.flag_for_review)

```

5.4.3 UNIT TESTS — ANALYTICS MODULE (ANALYTICS/TESTS.PY)

The following tests verify DBSCAN cluster formation on dense coordinate groups and noise point exclusion for isolated coordinates:

```

from django.test import TestCase
from django.contrib.auth import get_user_model
from django.utils import timezone
from datetime import timedelta
from cases.models import Case, Ward
from analytics.services import run_dbscan
from analytics.models import HotspotRun, HotspotCluster

User = get_user_model()

class AnalyticsDBSCANTests(TestCase):

    def setUp(self):
        # Create a test Ward
        self.ward = Ward.objects.create(
            ward_name="HSR Layout",
            city="Bengaluru",
            centroid_lat=12.9121,
            centroid_lon=77.6446
        )

        self.reporter = User.objects.create_user(
            email="reporter_an@pawmap.org",
            password="pwd",
            full_name="Reporter User",
            role="Reporter"
        )

```

```

        # Create 5 clustered cases close to each other (approx
within 100m)
        # Centered around (12.9121, 77.6446)
        for i in range(5):
            Case.objects.create(
                latitude=12.9121 + (i * 0.0001), # tiny offset
                longitude=77.6446 + (i * 0.0001),
                ward=self.ward,
                species="Dog",
                severity=3,
                reporter=self.reporter
            )

        # Create 2 noise cases far away
        Case.objects.create(
            latitude=12.9719, # Indiranagar lat
            longitude=77.6412,
            ward=self.ward,
            species="Cat",
            severity=1,
            reporter=self.reporter
        )

    def test_run_dbscan_creates_clusters(self):
        """Test DBSCAN groups clustered points and ignores noise
cases."""
        run, error = run_dbscan(eps_m=500, min_samples=5, days=30)

        self.assertNil = False
        self.assertIsNotNone(run)
        self.assertIsNone(error)

        # Verify HotspotRun metadata
        self.assertEqual(run.total_cases_processed, 6)

        # Verify exactly 1 cluster was created (noise of 1 case far
away is skipped)
        clusters = HotspotCluster.objects.filter(run=run)
        self.assertEqual(clusters.count(), 1)

        cluster = clusters.first()
        self.assertEqual(cluster.case_count, 5)
        self.assertEqual(cluster.dominant_species, "Dog")
        # Centroid lat/lon should be close to HSR Layout centroid
        self.assertAlmostEqual(float(cluster.centroid_lat),
12.9123, places=3)
        self.assertAlmostEqual(float(cluster.centroid_lon),
77.6448, places=3)

```

CHAPTER 6

CONCLUSION

6.1 DESIGN AND IMPLEMENTATION ISSUES

The development of PawMap surfaced several technical challenges that required deliberate design decisions to resolve. The most significant are documented below:

- **Race Condition on Case Claiming:** The initial implementation used optimistic locking with a version counter, but this still allowed a narrow window for duplicate claims under high concurrency. The solution was to switch to PostgreSQL row-level pessimistic locking via `SELECT FOR UPDATE` inside `transaction.atomic()`, which provides a hard serialisation guarantee at the database level. Tests confirmed that parallel claiming attempts consistently result in exactly-one success and one HTTP 409 Conflict.
- **Composite Foreign Key Limitation:** Django ORM does not natively support composite foreign keys. The `AnimalType` table uses a composite primary key (`species`, `breed`). This relationship was enforced at the model validation layer via a `clean()` override that calls `AnimalType.objects.filter(species=self.species, breed=self.breed).exists()`, rather than relying on a database-level FK constraint.
- **PostGIS GIST Index Performance:** Early spatial proximity queries (`ST_DWithin`) on `Case.location_geom` ran slowly without an explicit GIST index. Adding `CREATE INDEX CONCURRENTLY idx_case_location ON cases USING GIST(location_geom)` reduced volunteer dispatch query times from ~800ms to ~12ms on the 10,000-record pilot dataset.
- **DBSCAN Coordinate Units:** scikit-learn's DBSCAN with `metric='haversine'` requires input coordinates in radians, not degrees. An early implementation omitted the radians conversion, producing wildly incorrect cluster boundaries. The fix — `eps_rad = eps_m / 6371000` and `np.radians(coords)` — was verified against known Bengaluru ward centroids.
- **OneDrive File Lock (Development Environment):** During report generation, the target `.docx` file was locked by Microsoft Word's OneDrive synchronisation process, causing `PermissionError`. A fallback save path was implemented in the

generation script to write to PawMap_Final_Report_v3.docx when the primary path is locked.

6.2 ADVANTAGES AND LIMITATIONS

PawMap delivers measurable advantages over the existing informal rescue coordination ecosystem:

- **Atomic Dispatch:** Eliminates the duplicate volunteer response problem endemic to WhatsApp-based coordination, ensuring each case is attended by exactly one responder at a time while maintaining a reassignment queue for unattended cases.
- **Persistent Animal Identity:** The ANIM-xxxx persistent ID system ensures that rescued animals retain their medical history across all caregiver handovers, veterinary consultations, and foster placements — a capability absent from all existing platforms reviewed.
- **Data-Driven Resource Allocation:** DBSCAN hotspot clustering transforms accumulated sighting data into actionable spatial intelligence, enabling NGOs to target sterilisation drives, vaccination campaigns, and volunteer training at statistically identified high-density zones rather than relying on anecdotal knowledge.
- **Transparent Volunteer Accountability:** The daily VRS leaderboard provides NGO administrators with objective performance data for recognising high-performing volunteers and supporting those who are struggling through coaching or deactivation reviews.

The current implementation also has identifiable limitations:

- **Active Internet Dependency:** While offline case caching is specified, it is not fully implemented in the current prototype. Volunteers in low-connectivity urban areas may experience delayed dispatch notifications.
- **Reverse Geocoding Dependency:** Ward name assignment depends on Nominatim's availability. Rate-limiting by Nominatim's free tier can cause delays during bulk case seeding.
- **Synthetic Pilot Data:** The pilot dataset of 100 records is synthetically generated and does not capture the biases of actual reporting behaviour (e.g., reporter density correlated with smartphone ownership, not actual stray density).

DBSCAN results on the pilot data should be interpreted as functional validation, not epidemiological findings.

6.3 FUTURE ENHANCEMENTS

Based on the project experience and feedback from potential NGO stakeholders, the following enhancements are planned for post-academic development:

- **IoT and NFC Collar Tagging:** Integrating NFC reader functionality into the mobile application would allow volunteers to scan RFID collar tags attached to known strays, instantly pulling their medical history and last-known location without manual Animal ID entry.
- **AI-Based Injury Severity Classification:** A fine-tuned MobileNetV3 image classifier could replace the manual 1-5 severity selector, automatically estimating injury severity from the submitted photograph and pre-populating the form field.
- **Drone-Assisted Hotspot Surveys:** Drone flights over DBSCAN-identified high-density zones could provide automated animal headcounts, supplementing reporter-based data with sensor data.
- **Municipal API Integration:** A REST API integration with BBMP's Animal Birth Control tracking database would allow PawMap to annotate cases with sterilisation history, eliminating repeated intervention on the same animals.
- **Multilingual Support:** Adding Kannada, Tamil, Telugu, and Hindi translations of the mobile interface would significantly expand the accessible reporter base beyond English-literate urban communities.
- **WebSocket Real-Time Dashboard:** Replacing the polling-based NGO dashboard refresh with Django Channels WebSocket connections would deliver truly live case updates to the admin interface.

CHAPTER A

PILOT DATA DESCRIPTION

A.1 DATASET OVERVIEW AND FEASIBILITY STUDY

A synthetic pilot dataset of 100 rescue case records was generated to validate the DBSCAN clustering pipeline, test the coverage gap analysis against known volunteer footprints, and seed the platform for user acceptance testing. The dataset reflects realistic geographic and temporal distributions across 10 BBMP administrative wards in Bengaluru. Species distribution in the pilot dataset is: 60 Dog cases, 25 Cat cases, 10 Cow/Cattle cases, and 5 Other (bird, goat) cases, approximating the approximate real-world sighting distribution reported in Bhalla et al. (2021) [2].

The feasibility study conducted in SRS Section 7.1 confirmed that the proposed technology stack is appropriate for the project timeline. Django, React Native, PostgreSQL with PostGIS, and scikit-learn are mature open-source frameworks with active community support. The team's prior coursework exposure to Python, SQL, and React provided a sufficient foundation. Google Maps Platform and Firebase offered free-tier credits adequate for pilot-scale testing under 500 users. AWS Free Tier covered development and staging environments within the estimated \$30–50/month budget.

A.2 DATASET FIELDS AND DEFINITIONS

Table A.1

Table 32: Pilot Dataset — Field Definitions and Usage

Field Name	Type	Description	Used In Module
case_id	String	Unique CASE-xxxx sequential identifier	All modules
timestamp_utc	Datetime	UTC timestamp of report submission	Hotspot detection, VRS
latitude / longitude	Float	WGS84 GPS coordinates of sighting	DBSCAN, coverage gap, Leaflet map
ward_name	String	BBMP administrative ward (reverse-geocoded)	Coverage gap analysis, reports
species	ENUM	Dog, Cat, Cow, Bird, Other	Case filtering, hotspot disaggregation

estimated_age	ENUM	Puppy / Kitten / Calf / Juvenile / Adult / Senior	Case details, animal profile
severity	Int (1-5)	Reporter-assigned injury severity; 1=healthy, 5=critical	Volunteer dispatch priority, SOS trigger
aggression_level	Int (1-5)	Observer-assessed aggression level	Volunteer case card safety indicator
injury_type	Text	Free-text description of injury or condition	Medical module, case detail
bystander_action	ENUM	none / fed / contacted_ngo	Contextual reporting data
volunteer_id	String	ID of assigned volunteer (nullable before claim)	VRS computation, dispatch history
response_time_min	Float	Minutes between case creation and volunteer claim	VRS input: response time component
resolution_status	ENUM	Resolved, Escalated, Unresolved	Impact reporting, coverage gap analysis
animal_id	String	Linked ANIM-xxxx profile ID (assigned post-rescue)	Medical records, adoption module

A.3 SAMPLE WARD CENTROIDS USED IN PILOT DATASET

Table A.2

Table 33: BBMP Ward Centroids for Pilot Dataset Geographic Distribution

Ward Name	Centroid Latitude	Centroid Longitude	Cases Seeded
Koramangala	12.934800	77.618900	12
Indiranagar	12.971900	77.641200	11
Jayanagar	12.925000	77.593800	10
JP Nagar	12.906300	77.585700	10
HSR Layout	12.912100	77.644600	10
Whitefield	12.969800	77.750000	12
Malleshwaram	12.998200	77.568300	10
Hebbal	13.035800	77.597000	9
Yelahanka	13.100700	77.596300	8
Marathahalli	12.956900	77.701100	8

A.4 DBSCAN RUN RESULTS ON PILOT DATASET

The DBSCAN analytics job was executed on the 100-record pilot dataset with the parameters $\text{eps}=500\text{m}$, $\text{min_samples}=5$, haversine metric. The run detected 17 spatial clusters across the 10 seeded wards, with 8 noise points (isolated sightings that did not

form part of any cluster). Cluster sizes ranged from 5 to 13 cases. The dominant species across all clusters was Dog (14 of 17 clusters), consistent with the pilot dataset's 60% Dog case proportion. The analytics job completed in under 0.3 seconds on 100 records, confirming it will scale to the 10,000-record target within the 60-second NFR-04 constraint.

Table A.3

Table 34: DBSCAN Pilot Run Summary Statistics

Metric	Value
Total case records processed	100
DBSCAN eps parameter	500 metres (0.0000785 radians)
DBSCAN min_samples parameter	5
Total clusters detected	17
Noise points (label = -1)	8
Largest cluster (case count)	13
Smallest cluster (case count)	5
Dominant species (most clusters)	Dog (14 of 17 clusters)
Analytics job execution time	< 0.3 seconds on 100 records

CHAPTER B

REFERENCES

- [1] M. Ester, H. P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD-96), Portland, OR, USA, 1996, pp. 226–231.
- [2] S. J. Bhalla, R. Kemmers, and A. Vasques, "Stray appetites: A socio-ecological analysis of free-ranging dogs living alongside human communities in Bangalore, India," *Urban Ecosystems*, vol. 24, no. 6, pp. 1245–1258, 2021.
- [3] L. Kochumon et al., "The Paw Project: A Mobile Application for Identifying Stray Dogs," *International Journal of Engineering Research & Technology (IJERT)*, vol. 12, no. 5, 2023.
- [4] M. Rajwani et al., "Enhancing Animal Rescue Efficiency Using AI for Real-Time Reporting and Assistance," in Proc. Atlantis Press, 2025.
- [5] K. Subramaniam and H. K. Mammi, "Stray Rescue Management System (SRMS)," *International Journal of Innovative Computing*, vol. 15, no. 1, pp. 139–144, 2025.
- [6] H. S. Efe, G. Yildiz, and O. Yakut, "Be My Home in Smart Cities: A Mobile Application for Stray Animal Tracking, Adoption and Shelter Management," *European Journal of Natural and Social Sciences*, vol. 4, no. 1, 2025.
- [7] Karuna Nidhan, *Stray Animal Rescue Application*, 2025. [Online]. Available: <https://karunanidhan.app/> [Accessed: 21 Aug. 2026].
- [8] R. Rajkumar et al., "Paw Safe: An Extensive Technology-Driven Framework for Stray Dog Rescue, Healthcare Management, Community Engagement, and Smart Urban Governance," 2026.
- [9] World Health Organization and WSPA, *Guidelines for Dog Population Management*. Geneva: WHO Press, 2010.
- [10] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [11] A. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, vol. 2014, no. 239, 2014.
- [12] Django Software Foundation, "Django Documentation, Version 5.2," 2026. [Online]. Available: <https://docs.djangoproject.com/en/5.2/>
- [13] React Team, "React 18 Documentation," Meta Open Source, 2024. [Online]. Available: <https://react.dev/>
- [14] PostgreSQL Global Development Group, "PostgreSQL 15 Documentation," 2023. [Online]. Available: <https://www.postgresql.org/docs/15/>

-
- [15] PostGIS Development Group, "PostGIS 3.3 Manual," 2023. [Online]. Available: <https://postgis.net/docs/>